НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«Київський Політехнічний Інститут імені Ігоря Сікорського»
Кафедра Технічної кібернетики

ЗВІТНІСТЬ ПО ПРАКТИЧНИМ РОБОТАМ З ДИСЦИПЛІНИ «ПРОЕКТУВАННЯ АЛГОРИТМІВ»

Виконав студент
групи ЗПІ-зп-02
Кононов М. А.

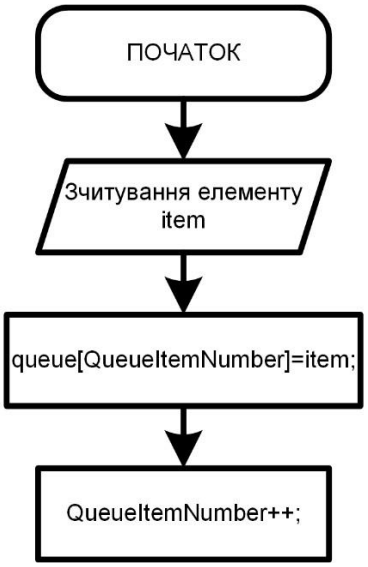
Мова програмування: C

Київ-2021

ПРАКТИЧНА РОБОТА №1. Абстрактні типи даних.

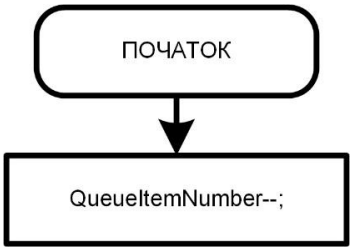
Постановка задачі: дослідження АТД «черга» та «двозв'язний лінійний список», реалізації за допомогою масивів та покажчиків на мові програмування C. Блок-схеми реалізацій «черги»:

Додавання елементу (масив)

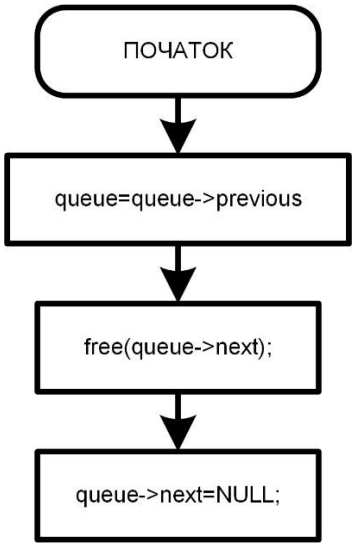


Додавання елементу (покажчик)

Видалення елементу (масив)



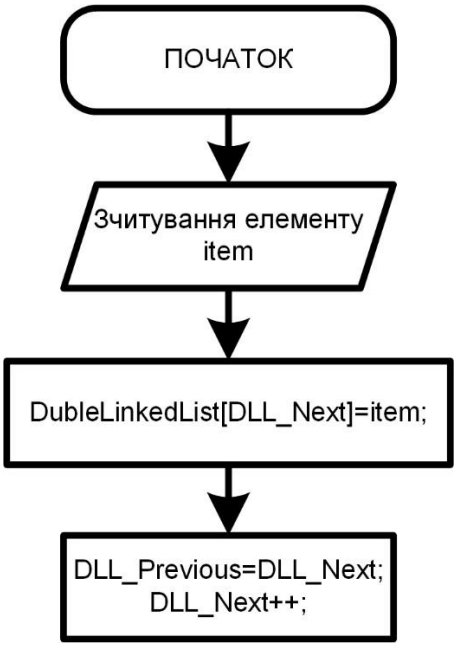
Видалення елементу (покажчики)



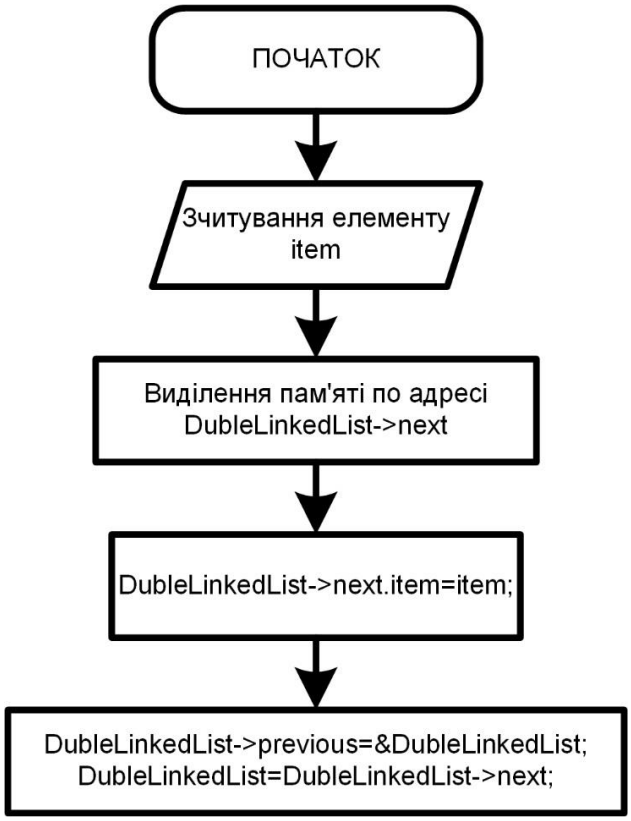
Складність усіх наведених алгоритмів складає O(1).

Блок-схеми реалізації АТД «двозв’язний лінійний список»:

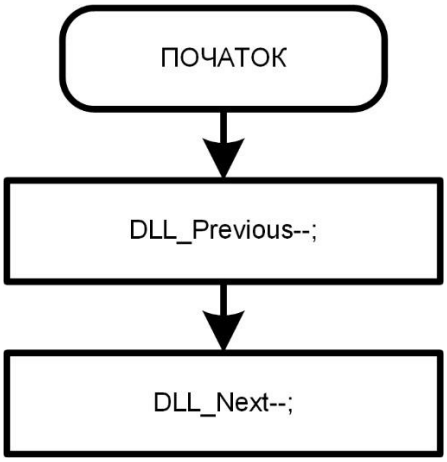
Додавання елементу (масив)



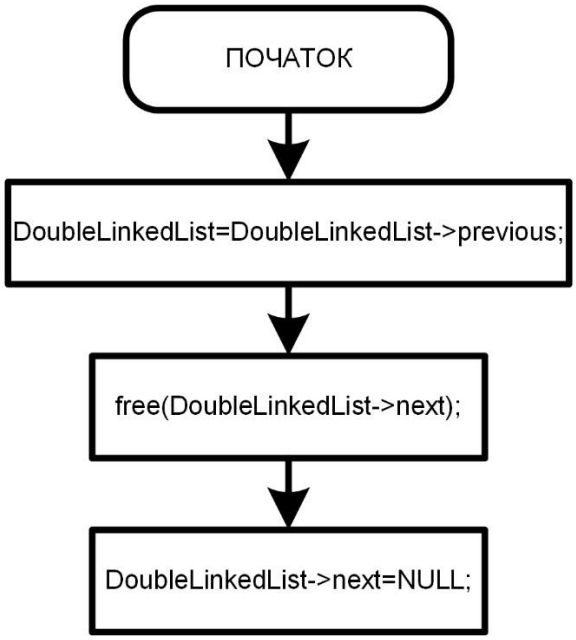
Додавання елементу (покажчик)

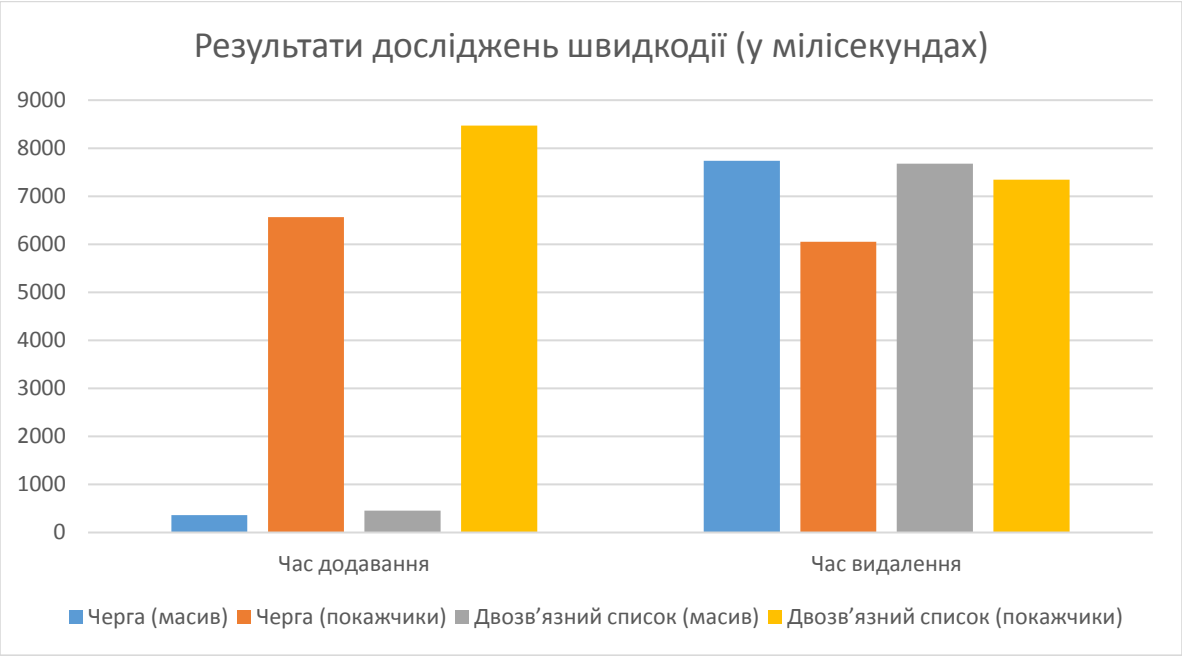


Видалення елементу (масив)



Видалення елементу (покажчики)





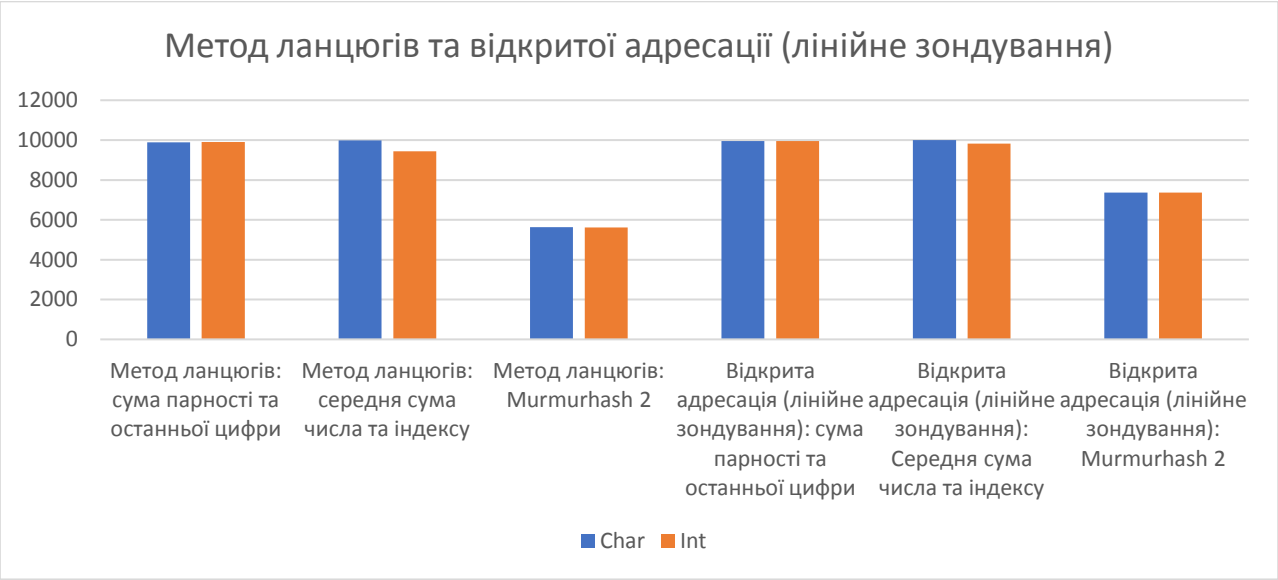
Висновок. Складність усіх наведених алгоритмів складає O(1). Тестові випробування виконувались над випадковими числами типу «*int*» у кількості 50 млн. Проаналізувавши швидкодію, ми дійшли висновку, що реалізації вибраних типів даних за допомогою масивів є найбільш швидкодіючими, потребують менше пам'яті та є простими з точки зору написання коду. Найбільш вибагливою до ресурсів процесора є операція видалення, оскільки реалізований нами алгоритм змінює розмір виділеної оперативної пам'яті. Вказані оцінки складності алгоритмів є аналітичними та, мабуть, повністю відповідають реальним результатам наведеного дослідження.

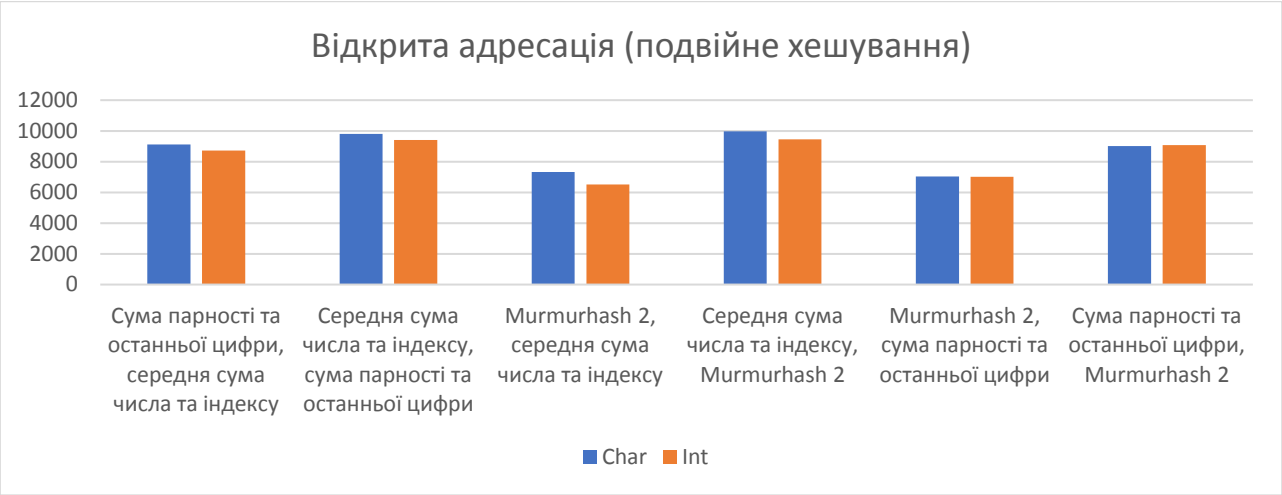
ПРАКТИЧНА РОБОТА №2. Пошук у масиві. Закрите хешування.

1. Постановка задачі: дослідження трьох способів вирішення колізій при хешуванні: метод ланцюгів, відкрита адресація (лінійне зондування), відкрита адресація (подвійне хешування) з використанням трьох хеш-функцій: сума парності та останньої цифри, середня сума числа та його індексу в масиві, авторська функція Murmurhash 2. Усі випробування здійснювались на масивах з даними типів *int* та *char* (у хеш-таблицю додавались масиви випадкових даних однакового розміру). Результати випробувань з наступними параметрами:

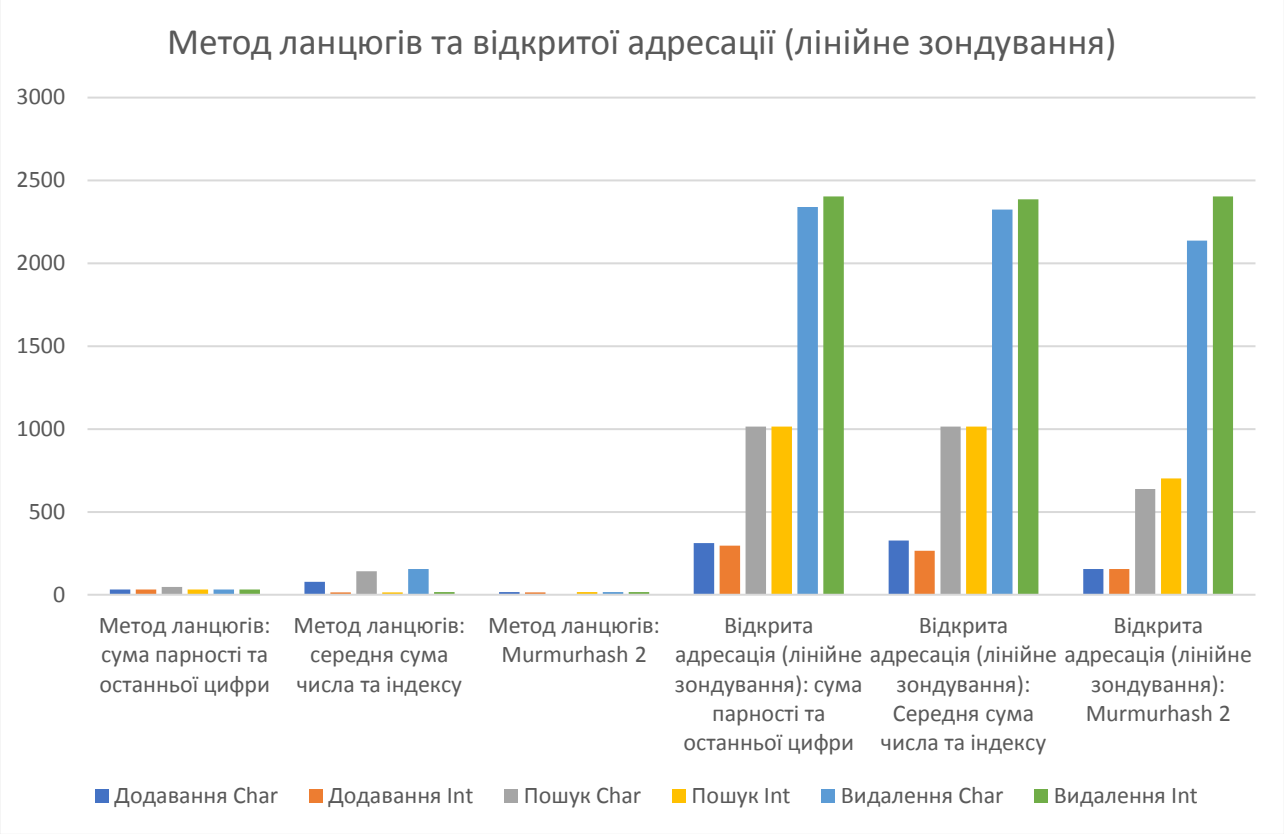
Розмір одного елементу таблиці	100 чисел
Кількість елементів, над якими здійснювались операції додавання, пошуку та видалення	10000
Розмір хеш-таблиці (максимальна кількість елементів у ній)	10000

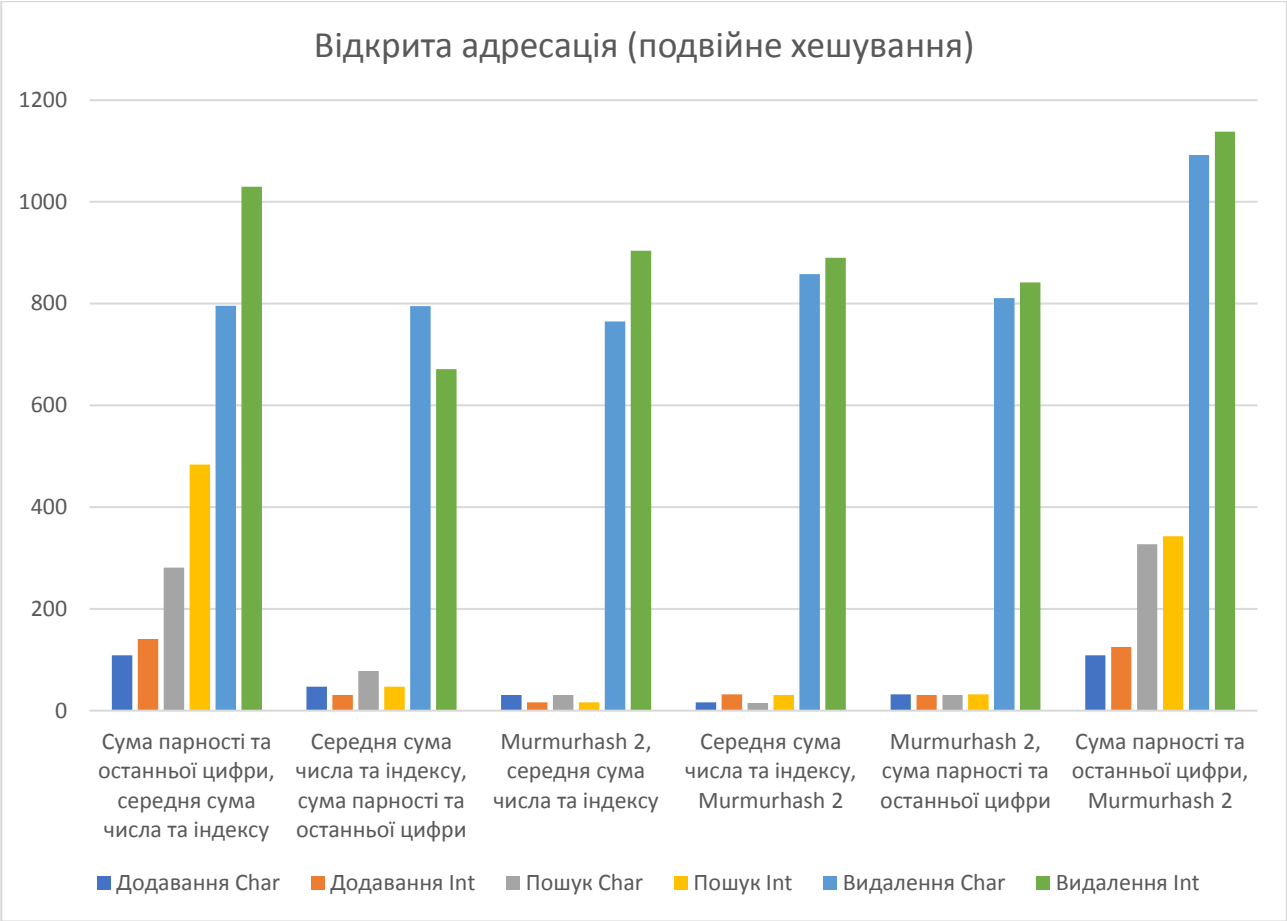
Порівняння кількості колізій



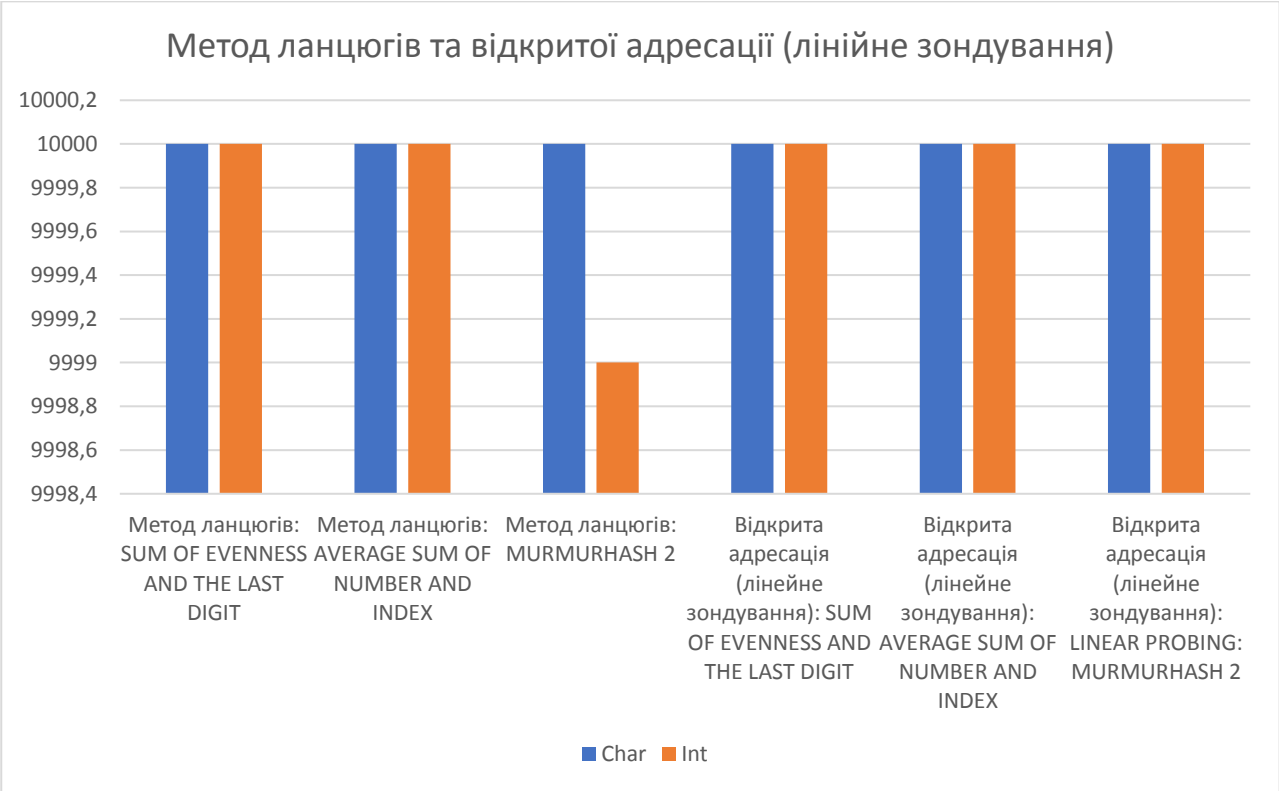


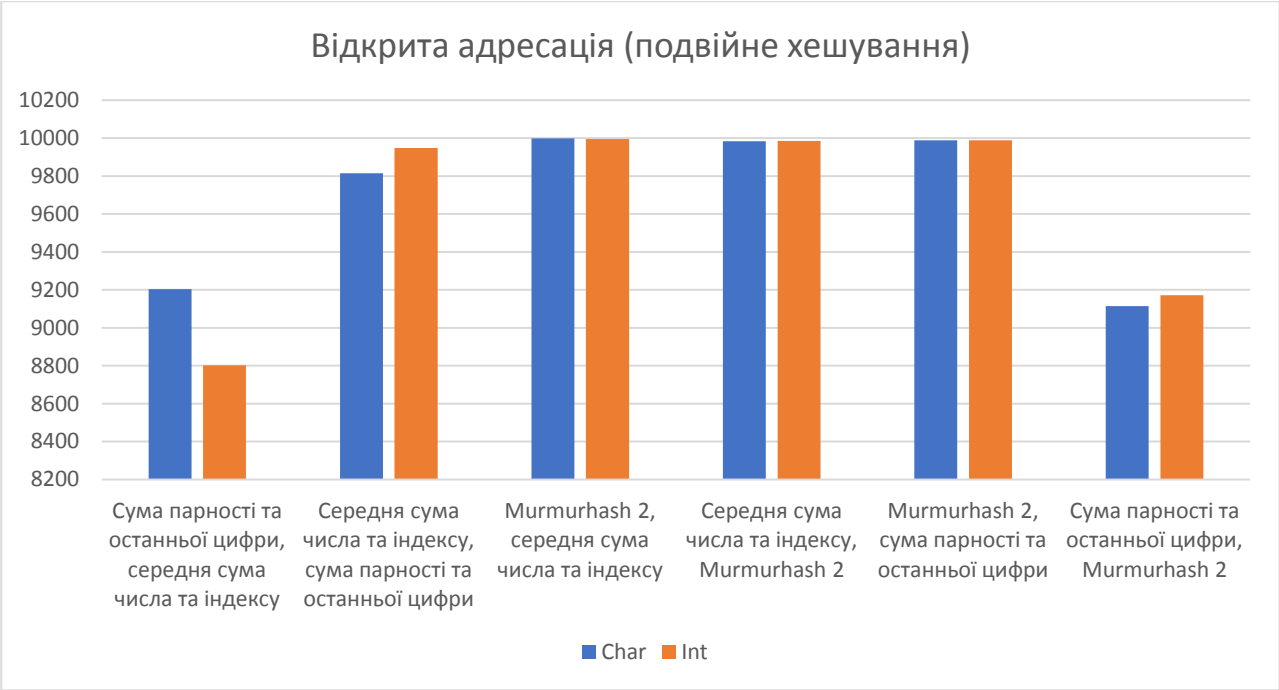
Порівняння часових витрат на виконання всіх операцій над усіма елементами (у мілісекундах)





Порівняння кількості доданих елементів (операції вдалого пошуку)





2. Висновок.

Проаналізувавши три методи вирішення колізій з виконанням трьох хеш-функцій над масивами двох типів даних *int* та *char*, ми дійшли висновку, що найбільш стійкою до колізій є хеш-функція Murmurhash 2, розроблена Остином Епплбі. На другому місті – сума парності та останньої цифри, найбільшу кількість колізій викликає функція обчислення середньої суми числа та його індексу в масиві. Для збільшення кількості доданих елементів після виконання кожної хеш-функції циклічно виконувалась операція ділення на 10 доки значення хешу не стало меншим за максимальний розмір хеш-таблиці для збільшення ймовірності додавання елементу саме за обчисленою адресою. Ефективність (швидкодія виконання всіх операцій та стійкість до колізій) розроблених нами рішень залежить від заданих параметрів: розміру масивів, їх кількості, максимального розміру хеш-таблиці. Найбільш вибагливою до часу та ресурсів процесора є операція видалення при використанні методів відкритої адресації (лінійного зондування та подвійного хешування), оскільки після неї виконується зміщення багатьох доданих елементів на найкращу для них позицію.

При використанні методів ланцюгів та відкритої адресації засобом лінійного зондування хеш-таблиця може бути заповнена на 100%, тому

реструктуризацію потрібно проводити лише в тому випадку, якщо кількість елементів, які ми бажаємо додати, перевищує максимальний заданий розмір хеш-таблиці. Метод відкритої адресації засобом подвійного хешування не дає можливість відслідкувати коли потрібно збільшити розмір, оскільки можливість додавання елементу повністю залежить від значень обох хеш-функцій. Для більш точної оцінки швидкодії роботи всіх алгоритмів ми не реалізували можливість зупинки циклічного виконання операції додавання у випадку відсутності місця для нових елементів та сигналізацію про необхідність реструктурувати дані. Якщо елемент не може бути доданий з причини перевищення значення хеш-функції заданого розміру таблиці або повного заповнення таблиці при використанні методів відкритої адресації, алгоритм просто збільшить лічильник недоданих елементів.

3. Відповіді на контрольні запитання.

3.1. Хеш-функція, або хеш-функція – функція, що перетворює вхідні дані будь-якого (як правило великого) розміру в дані фіксованого розміру.

Хешування (хешування, англ. *hashing*) – перетворення вхідного масиву даних довільної довжини у вихідний бітовий рядок фіксованої довжини. Такі перетворення також називаються хеш-функціями, або функціями згортання, а їхні результати називають хешем, хеш-кодом, хеш-сумою, або дайджестом повідомлення.

3.2. При закритому хешуванні в таблиці сегментів зберігаються безпосередньо елементи таблиці, а не заголовки списків. Тому в кожному сегменті може зберігатися тільки один елемент словника. При закритому хешуванні застосовується методика повторного хешування. Якщо ми спробуємо помістити елемент x в сегмент з номером h(x), який вже зайнятий іншим елементом (така ситуація називається колізією), то відповідно до методики повторного хешування вибирається послідовність інших номерів сегментів h1(x), h2(x),…, куди можна помістити елемент x. Кожне з цих місцеположень послідовно перевіряється, поки не буде знайдено вільне. Якщо

вільних сегментів немає, то, отже, таблиця заповнена і елемент x вставити не можна.

Дії при закритому хешуванні залежать від його типу. Ми розглянули два популярних методи закритого хешування (відкритої адресації): лінійне зондування та подвійне хешування. В обох випадках при виконанні всіх операцій (додавання, пошуку та видалення) спочатку перевіряється наявність якогось елементу за адресою значення хеш-функції. При лінійному зондуванні перед виконанням необхідних дій у випадку колізії ми рухаємось в обидва боки від найкращої позиції для елементу доки не знайдемо вільну комірку (при додаванні) або необхідний елемент (при пошуку та видаленні). При подвійному хешуванні, якщо за необхідною адресою вже є дані, ми обчислюємо значення додаткової хеш-функції та циклічно виконуємо операцію *(key1+i*key2)%HashTableSize* доки не знайдемо комірку зі значенням NULL (при додаванні) або бажаний елемент (при пошуку та видаленні). Видалення елементу при відкритій адресації призводить до оптимізації хеш-таблиці шляхом переміщення елементів, для яких раніше при додаванні розв’язувалась колізія, в комірки з оптимальними для них індексами (за значенням обчисленої хеш-функції), що в майбутньому значно прискорить їх пошук.

ПРАКТИЧНА РОБОТА №3. Основні типи двійкових дерев.

1. Постановка задачі: дослідження бінарного та червоно-чорного дерев з використанням різних типів вхідних даних (випадкові, з різним ступенем відсортованості та різною кількістю однакових значень), порівняння швидкодії реалізації основних операцій з абстрактними типами даних «черга» та «двозв'язний лінійний список».

Схема алгоритму додавання елементу в бінарне дерево

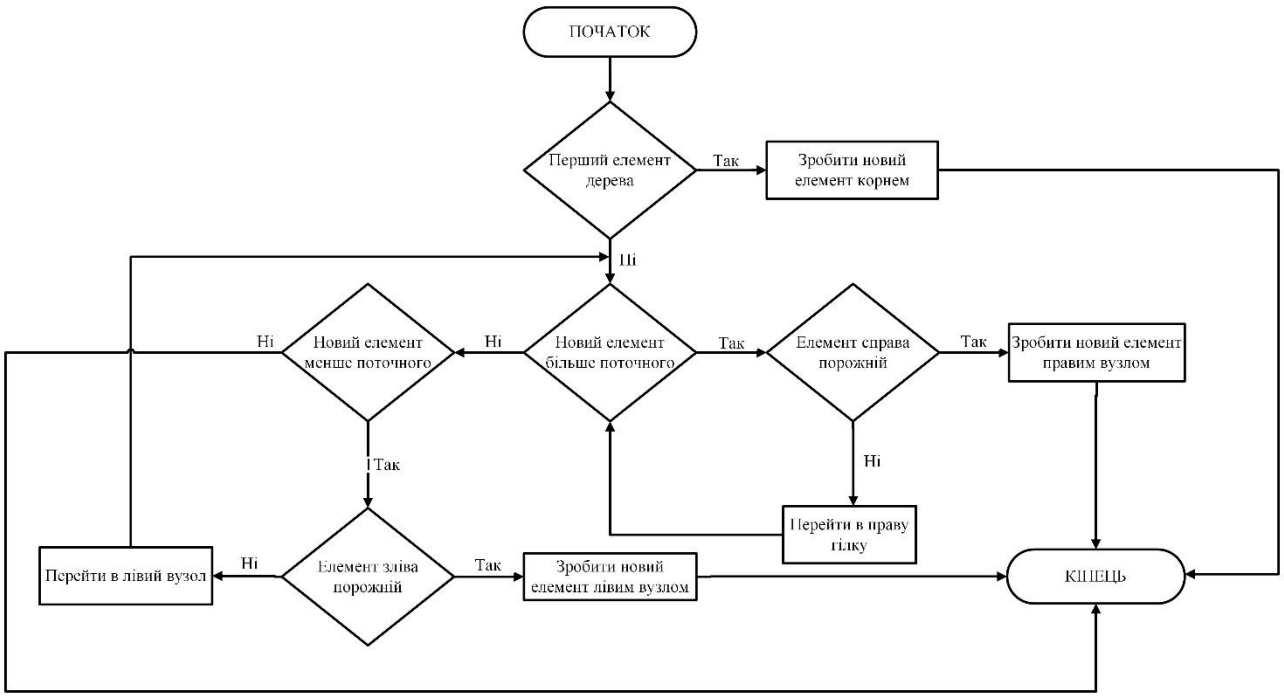
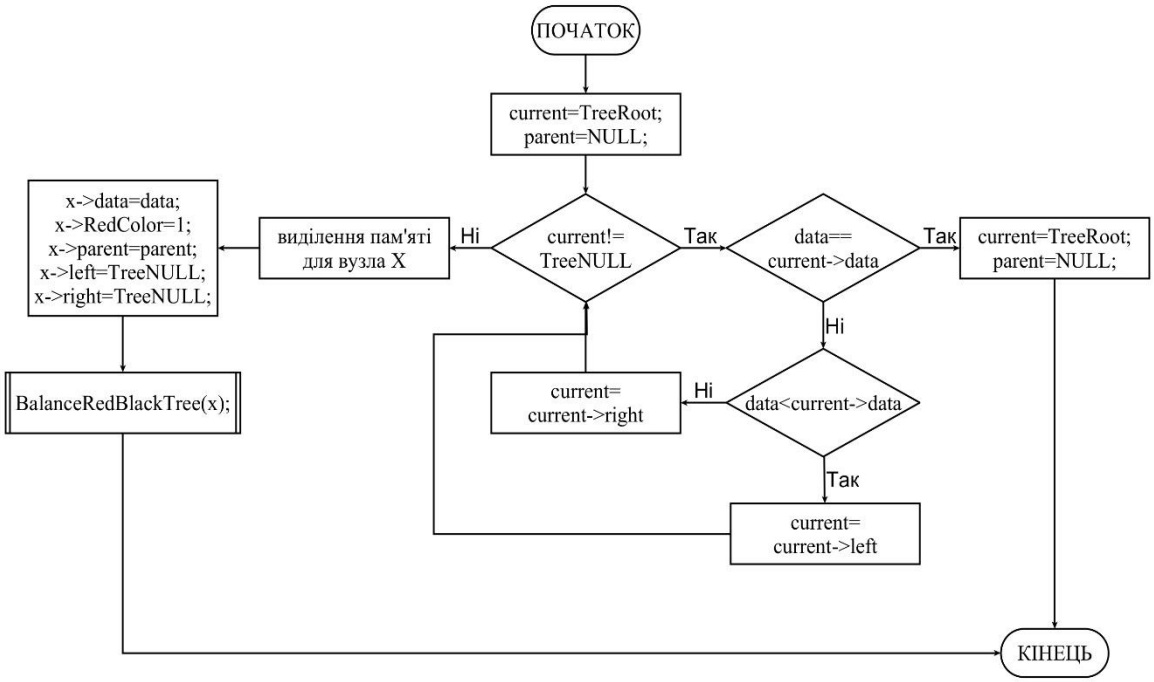
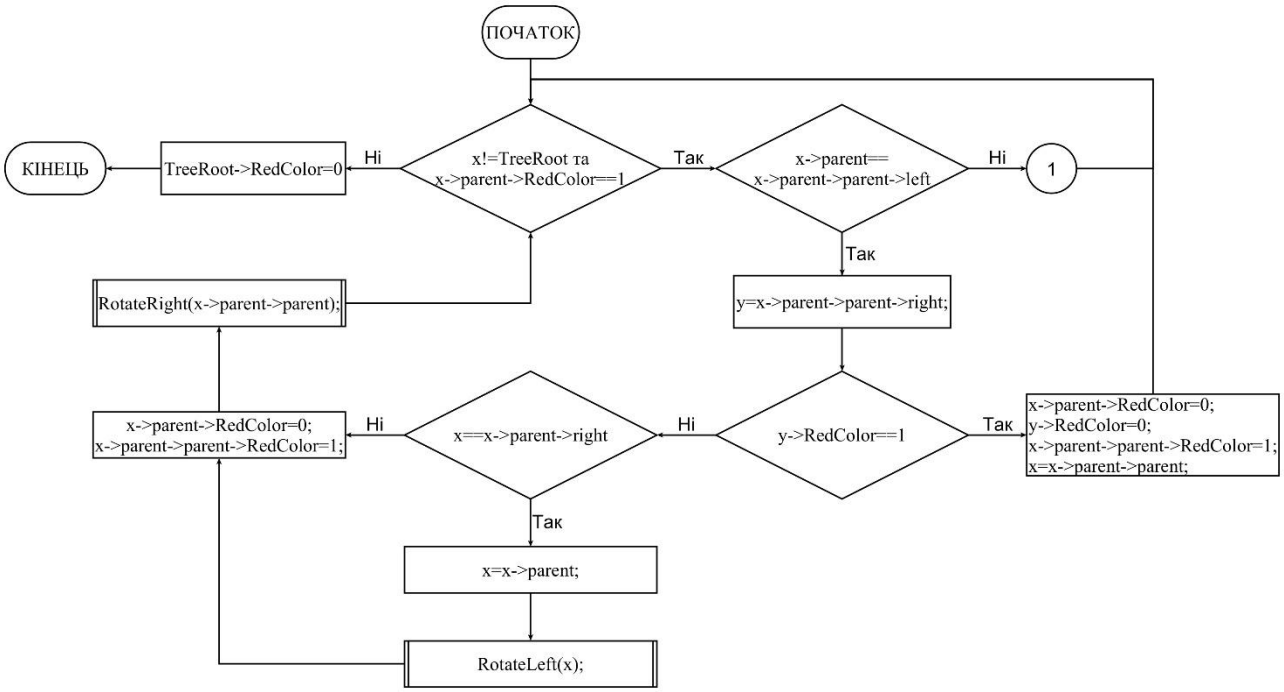
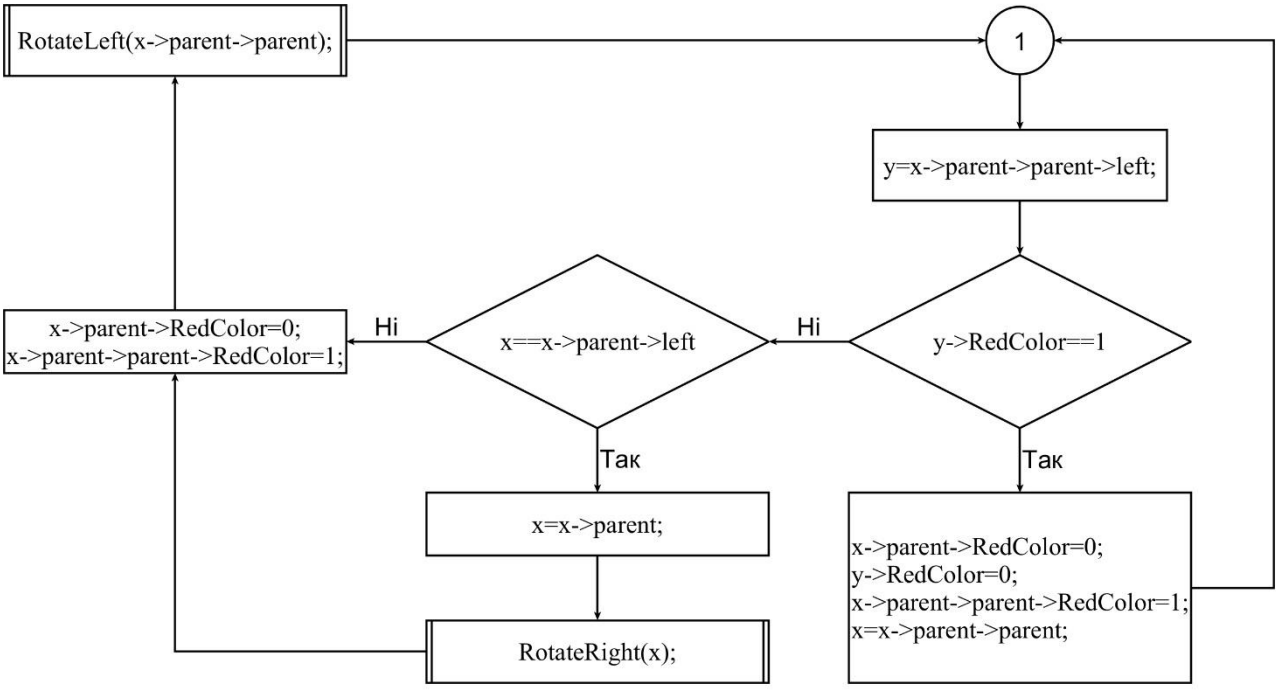


Схема алгоритму додавання елементу в червоно-чорне дерево

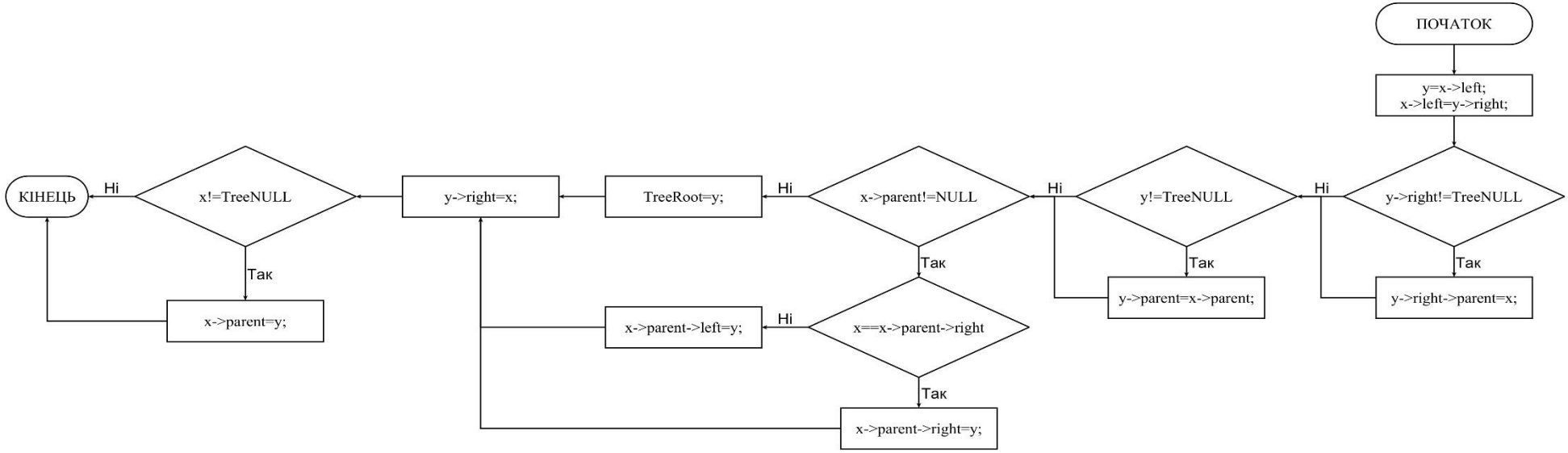


Балансування червоно-чорного дерева

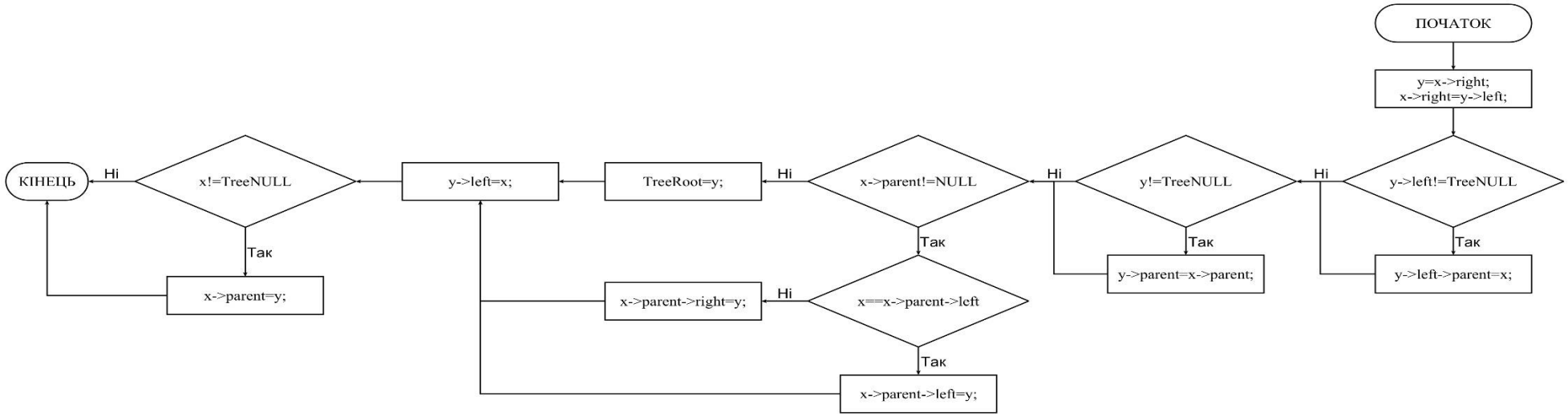




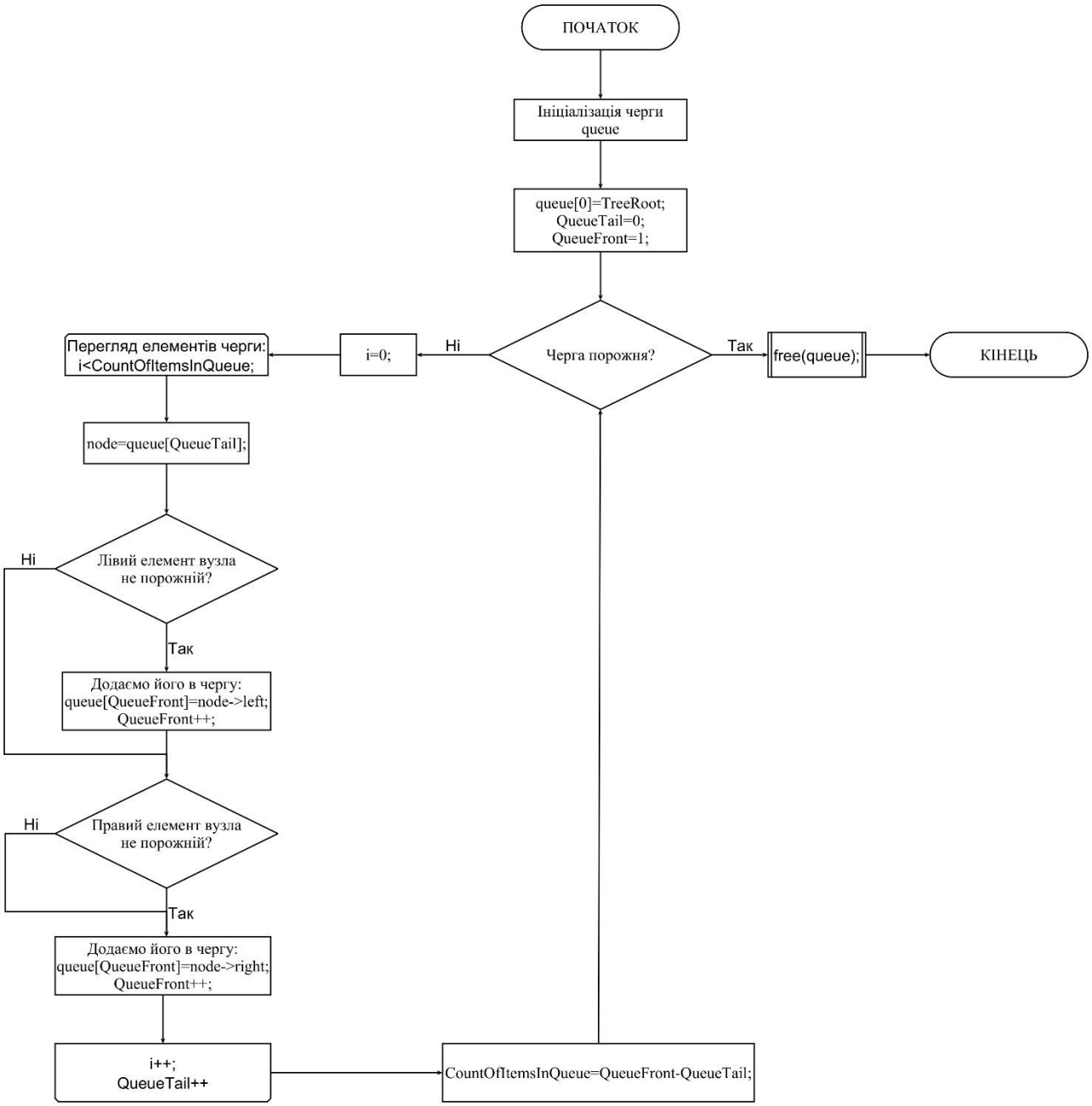
Алгоритм ротації вузлів вправо



Алгоритм ротації вузлів вліво



Нерекурсивний алгоритм обходу бінарного та червоно-чорного дерев



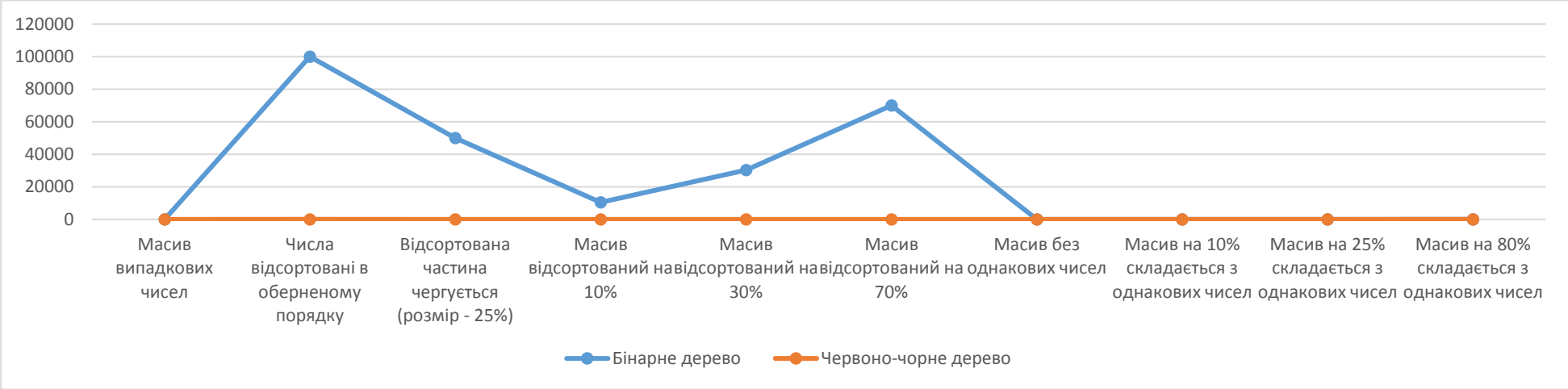
Складність розроблених алгоритмів

Назва	Випадок	Витрата пам’яті	Пошук	Вставка	Видалення
Бінарне дерево	Середній	O(n)	O(log n)	O(log n)	O(log n)
	Найгірший	O(n)	O(n)	O(n)	O(n)
Червоно-чорне дерево	Середній	O(n)	O(log n)	O(log n)	O(log n)
	Найгірший	O(n)	O(log n)	O(log n)	O(log n)

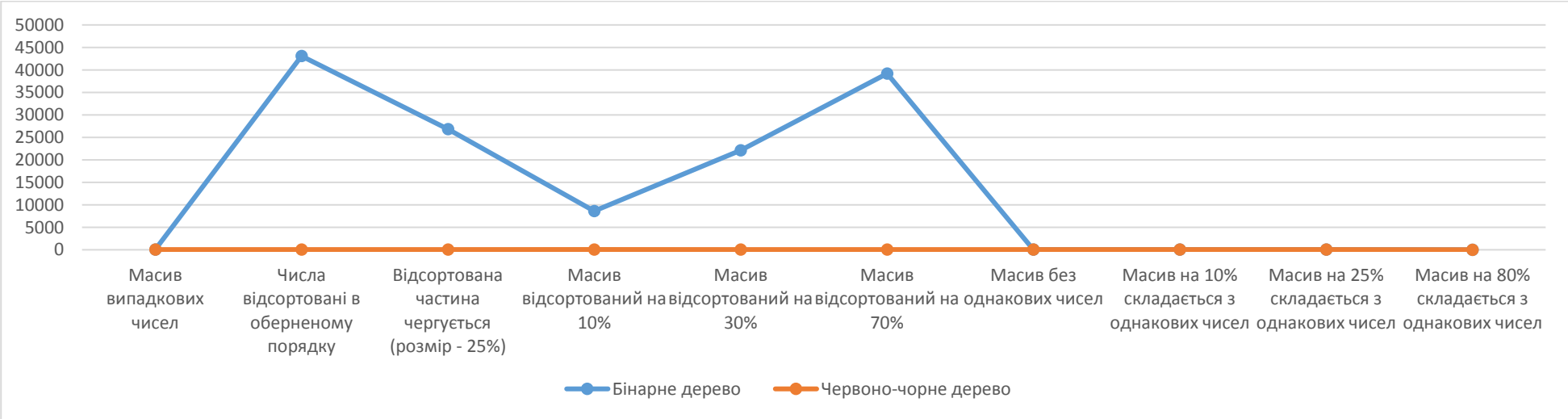
Порівняння ефективності бінарного та червоно-чорного дерев при кількості елементів 100000

(час вказаний у мілісекундах)

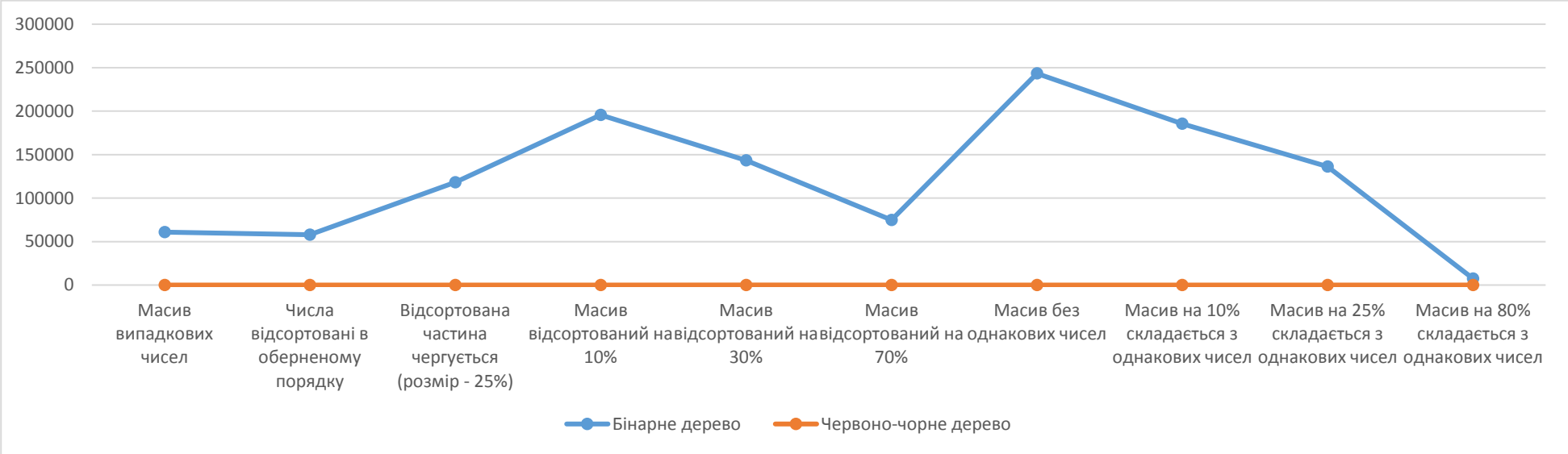
Висота дерев (максимальна довжина списків)



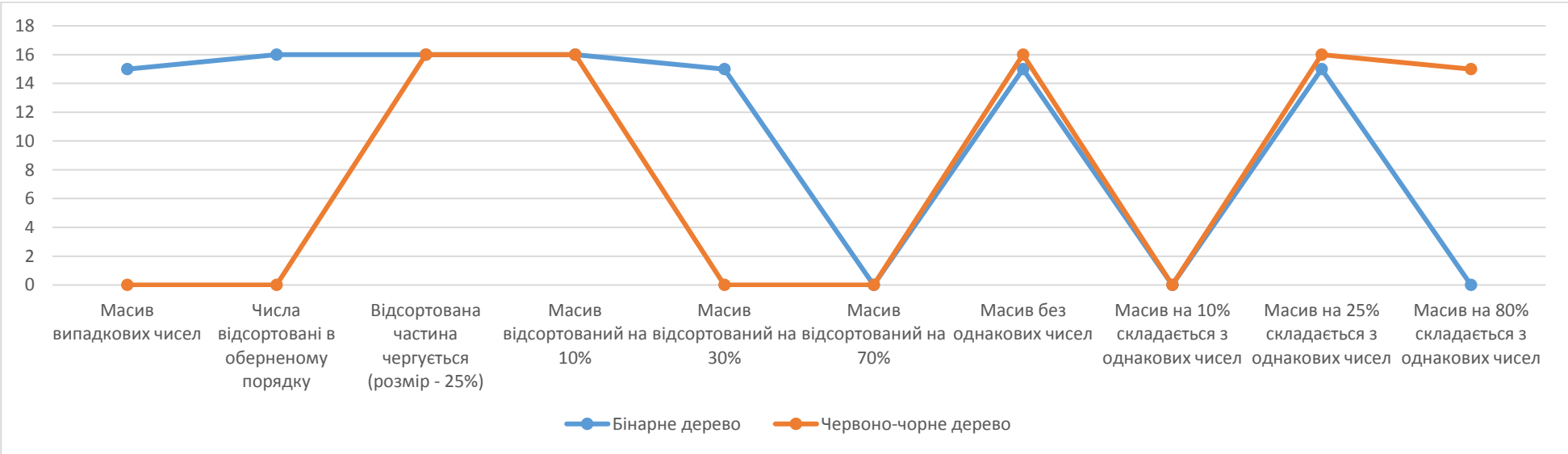
Час додавання всіх елементів



Час пошуку всіх елементів

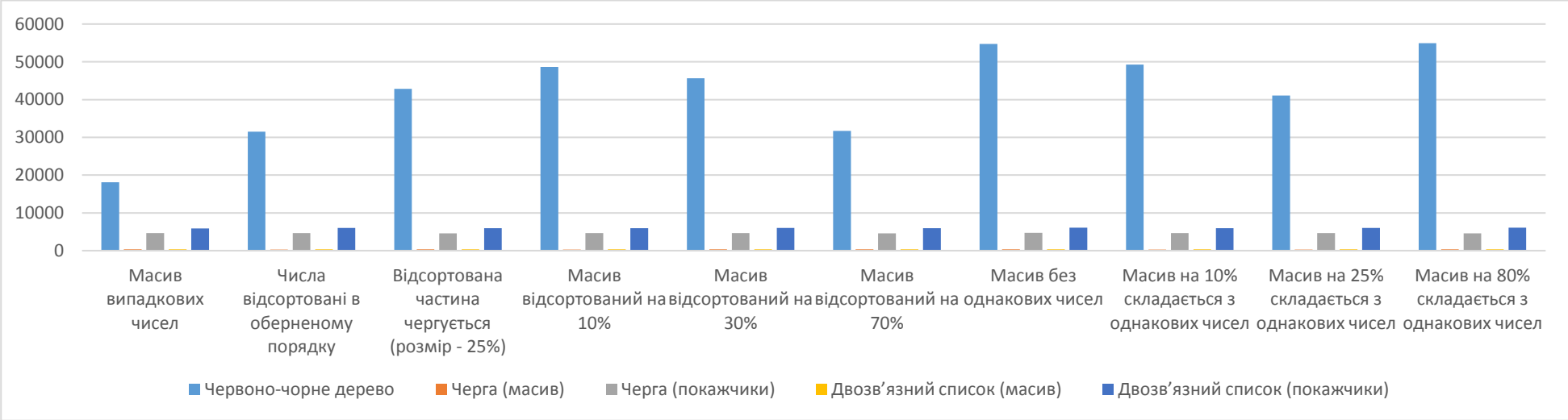


Час звільнення пам’яті

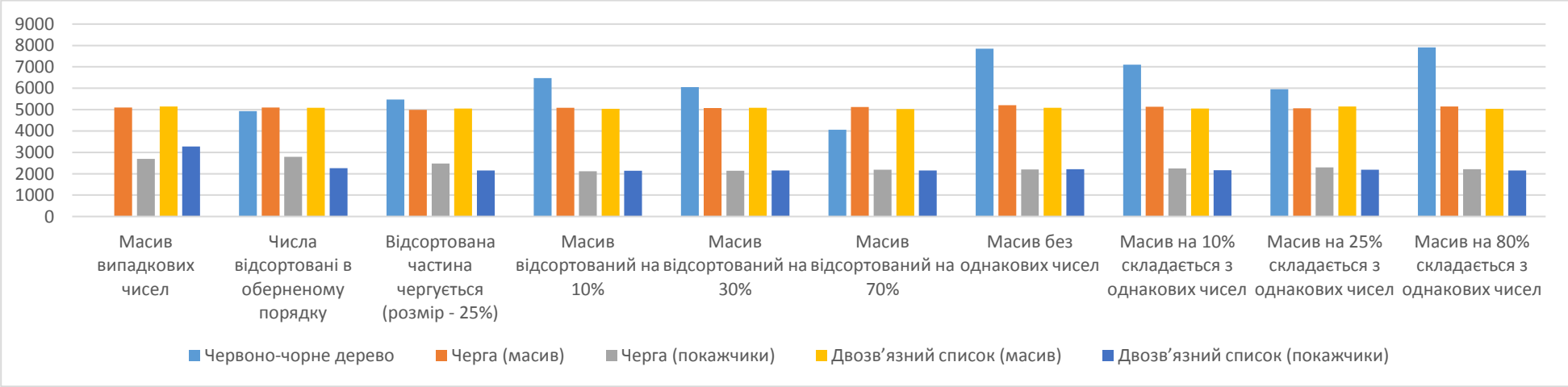


Порівняння ефективності червоно-чорного дерева, черги та двозв’язного лінійного списку з використанням даних різного характеру в кількості 50 млн чисел (час вказаний у мілісекундах)

Операція додавання



Операція видалення



4. Висновок. Бінарні та червоно-чорні дерева доцільно використовувати для збереження лише оригінальних (не однакових даних). У процесі виконання даної роботи нами було помічено, що рекурсивні алгоритми виконання основних операцій з бінарними деревами (додавання, пошук та звільнення виділеної пам'яті) спричинюють переповнення стеку викликів операційної системи та некоректне завершення програми при спробі обробки приблизно 50000 елементів. Тому ми реалізували нерекурсивні рішення, що підвищило їх продуктивність та уможливило обробку значно більшого обсягу чисел.

Порівнявши швидкодію виконання операції додавання в бінарне та червоно-чорне дерева ми помітили, що найбільш швидкодіючою структурою даних є друга. Згідно з вище наведеним графіком, швидкість виконання операції додавання в бінарне дерево залежить від характеру даних: відсортовані числа додаються швидше, а однакові опускаються. Було помічено, що червоно-чорне дерево є найменш залежним від типу вхідних даних при виконанні операцій додавання та звільнення виділеної пам'яті. Висота дерев, збалансованість та характер даних в них можуть впливати на швидкодію пошуку. Червоно-чорне дерево при будь-яких даних, навіть відсортованих, є значно нижчим за бінарне, що дозволяє знаходити інформацію за найменшу кількість кроків.

В рамках даної роботи було виконано порівняння ефективності червоно-чорного дерева та абстрактних структур даних «черга» і «двозв'язний лінійний список» на значно більшій кількості елементів, ніж попередні дослідження. Подібне випробування не виконувалось з бінарним деревом через дуже великі часові витрати на додавання елементів. Було встановлено, що червоно-чорне дерево є найменш швидкодіючим у порівнянні з названими структурами даних.

5. Відповіді на контрольні запитання.

А) Перерахуйте та прокоментуйте переваги та недоліки 2-3-4-дерев. Поясніть основні принципи їх побудови.

2-3-4 дерево (або 2-4 дерево) представляє собою самобалансовну структуру даних, яка зазвичай використовується для реалізації словників

(асоціативних масивів). Цифри означають дерево, де кожний вузол з дочірніми вузлами (внутрішній вузол) мають два, три або чотири дочірніх вузли. Властивості:

- Усі нелистові вершини мають одне поле даних і два піддерева або два поля і три піддерева;
- Усі нелистові вершини знаходяться на одному рівні (на нижчому) і мають один або два поля даних;
- Усі дані відсортовані (по принципу двійкового дерева пошуку);
- Поле в 2-вершині, як і в двійковому дереву пошуку, ділить простір можливих значень на два діапазони: (-∞, a) та [a_2, + ∞);
- Поля в 2 3-вершині ділять простір можливих значень на три діапазони: (-∞, a), [a, b) та (b, +∞).

Нелистові вершини містять одне або два поля, які вказують на діапазон значень в їх піддеревах. Значення першого поля завжди більше найбільшого значення в лівому піддереві та менше або дорівнює найменшому значенню в правому піддереві (або в центральному піддереві, якщо це 3-вершина). Аналогічним чином, значення другого поля (якщо воно є) завжди більше найбільшого значення в центральному піддереві та менше або дорівнює найменшому значенню в правому піддереві. Ці нелистові вершини використовуються для направлення функції пошуку до потрібного піддерева та шуканого листа. Рроро – р

Б) Перерахуйте та прокоментуйте переваги та недоліки червоно-чорних дерев. Поясніть основні принципи їх побудови.

Червоно-чорне дерево – різновид самозбалансованого бінарного дерева пошуку, вершини якого мають додаткові властивості (RB-властивості), зокрема «колір» (червоний або чорний). Ці біти кольору використовуються для забезпечення того, щоб дерево залишалося приблизно збалансованим при виконанні операцій вставки та видалення.

Червоно-чорні дерева – різновид збалансованих дерев, в яких за допомогою спеціальних трансформацій гарантується, що висота дерева h не

буде перевищувати O(log n). Зважаючи на те, що час виконання основних операцій на бінарних деревах (пошук, видалення, додавання елементу) є O(h), ці структури даних на практиці є набагато ефективнішими, ніж звичайні бінарні дерева пошуку. Основні властивості:

- кожна вершина або червона, або чорна
- корінь дерева – чорний
- кожний лист (NIL) – чорний
- якщо вершина червона, обидві її дочірні вершини чорні (інакше, батько червоної вершини – чорний)
- усі прості шляхи від будь-якої вершини до листів мають однакову кількість чорних вершин.

Такі властивості надають червоно-чорному дереву додаткового обмеження: найдовший шлях з кореня до будь-якого листа перевищує найкоротший шлях не більше ніж вдвічі. В цьому сенсі таке дерево можна назвати збалансованим. Зважаючи на те, що час виконання основних операцій з бінарними деревами пошуку залежить від висоти, таке обмеження гарантує їхню ефективність в найгіршому випадку, чого звичайні бінарні дерева гарантувати не можуть.

Для того, щоби зрозуміти, чому перелічені властивості забезпечують існування такого обмеження, зазначимо, що в червоно-чорному дереві, відповідно до властивості 4 не існує такого шляху, на якому б зустрілись дві червоні вершини підряд. Найкоротший шлях складається з усіх чорних вершин, а в найдовшому червоні та чорні вершини чергуються. З врахуванням властивості 5, отримуємо, що глибина будь-яких двох листів відрізняється не більше ніж вдвічі.

В деяких зображеннях червоно-чорних дерев, NIL-листки не наводяться, тому що вони не містять корисної інформації, але їхнє існування необхідне для забезпечення усіх властивостей.

Принципи побудови (вставка елементу):

Процедура починається аналогічно вставці елементу в бінарне дерево пошуку та фарбування її у червоний колір. Подальші дії залежать від кольорів сусідніх вершин. Зазначимо, що:

- властивість 2 завжди зберігається;
- властивість 3 порушується тільки при вставці червоної вершини, перефарбуванні чорної вершини в червону або обертанні;
- властивість 4 порушується тільки при вставці чорної вершини, перефарбуванні червоної вершини в чорну або обертанні.

На допоміжних діаграмах, вершина, яка додається, позначена N, первісний батько цієї вершини позначений P, батько вершини P («дідусь» N) позначений G. «Дядько» N (тобто вершина, яка має спільного з P батька – G) позначений як U. Розглянемо такі випадки:

Випадок 1: Нова вершина знаходиться в корені дерева. В такому випадку необхідно пофарбувати її в чорний колір для забезпечення властивості 1. Очевидно, що властивість 5 при цьому залишається справедливою.

Випадок 2: Батько нової вершини є чорним. Властивість 3 не порушена. В цьому випадку дерево є коректним. Властивість 5 також зберігається, адже червона вершина додається на місце чорного листа і це не змінює кількості чорних вершин на цьому шляху.

Випадок 3: Батько та дядько доданої вершини є червоними. Тоді ми можемо перефарбувати їх обох в чорний, а також перефарбувати дідуся в червоний. Тепер наша червона вершина має чорного батька. Завдяки тому, що будь-який шлях через батька чи дядька повинен проходити і через дідуся, кількість чорних вершин на шляху залишається незмінною. Однак батько дідуся (тобто прадідусь) може бути червоною вершиною, як тепер і дідусь. Якщо це так, слід повторити цю операцію рекурсивно.

Зауваження: В наступних випадках ми припускаємо, вершина P є лівою дитиною G. Якщо P – права дитина, ліво та право в аналізі наступних випадків слід поміняти місцями.

Випадок 4: Батько є червоним, але дядько – чорний. До того ж, нова вершина – права дитина свого батька, і батько в свою чергу ліва дитина свого батька. В такому випадку, ми можемо провести лівий поворот, внаслідок якого нова вершина та її батько поміняються ролями. Подальші дії з колишнім батьком проводяться відповідно до випадку 5. Зважаючи на те, що нова вершина та її батько є червоними, операція повороту не змінює умови 4.

Випадок 5: Батько є червоним, але дядько чорний. До того ж, нова вершина – ліва дитина свого батька, і батько є лівою дитиною свого батька. В такому випадку ми проводимо правий поворот навколо дідуся. В результаті колишній батько тепер є батьком і нової вершини, і свого колишнього дідуся. Ми знаємо, що колишній дідусь є чорним, інакше би батько не був червоним. Тепер варто поміняти місцями кольори колишнього батька та дідуся, і тепер дерево виконує властивість 3. Легко бачити, що властивість 4 також залишається незмінною.

В) Перерахуйте та прокоментуйте переваги та недоліки навантажених дерев. В яких випадках їх використання найефективніше?

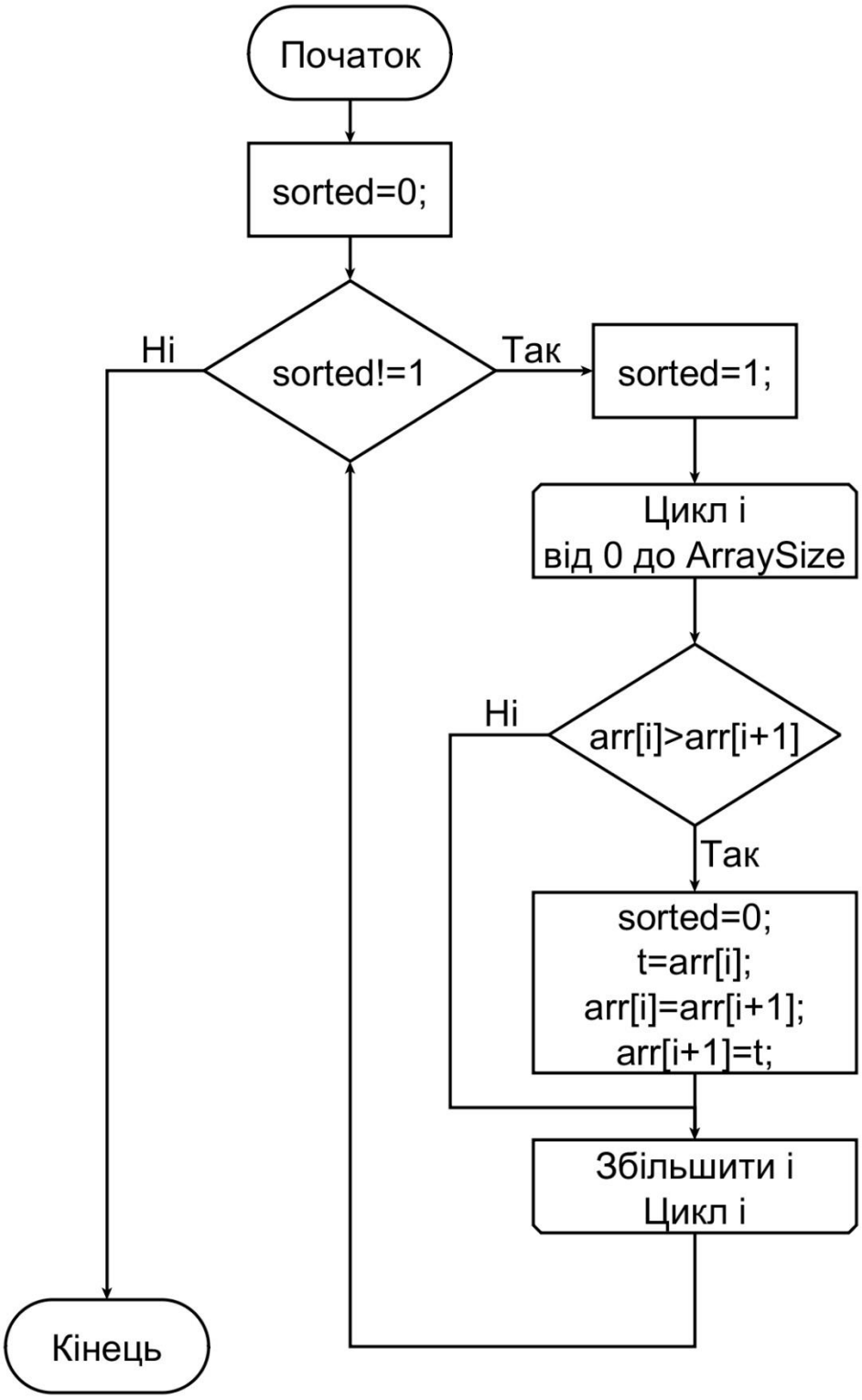
Префіксне (навантажене) дерево – структура даних, дерево, в якому шлях від кореня до листа визначає рядок. Рядки з однаковими префіксами мають спільний шлях від кореня довжиною цього префікса.

Префіксне дерево дозволяє зберігати асоціативний масив, ключами якого є рядки. На відміну від бінарних дерев, в листі дерева не зберігається ключ. Значення ключа можна набути прогляданням всіх батьківських вузлів, кожний з яких зберігає один або кілька символів алфавіту. Корінь дерева пов'язаний з порожнім рядком. Таким чином, нащадки вузла мають загальний префікс, звідки і відбулася назва даного абстрактного типу даних. Значення, пов'язані з ключем, зазвичай не пов'язані з кожним вузлом, а тільки з листям і, можливо, деякими внутрішніми вузлами.

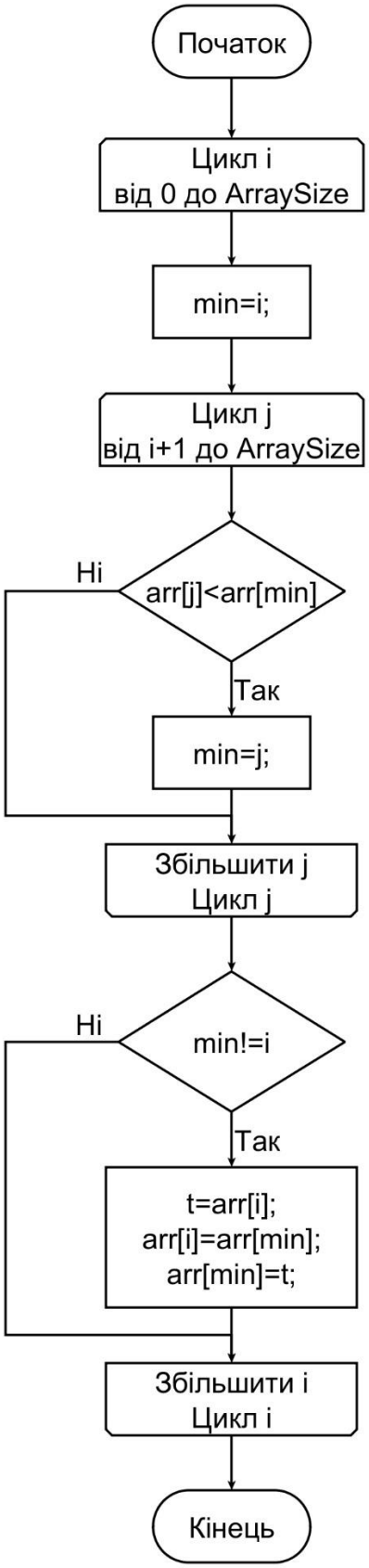
ПРАКТИЧНА РОБОТА №4. Алгоритми сортування. Методи сортування малих обсягів даних.

Постановка задачі: дослідження алгоритмів сортування методами бульбашки, вибору, простих вставок, підрахунку, порівняння їх із рекурсивним та нерекурсивним алгоритмами швидкого сортування.

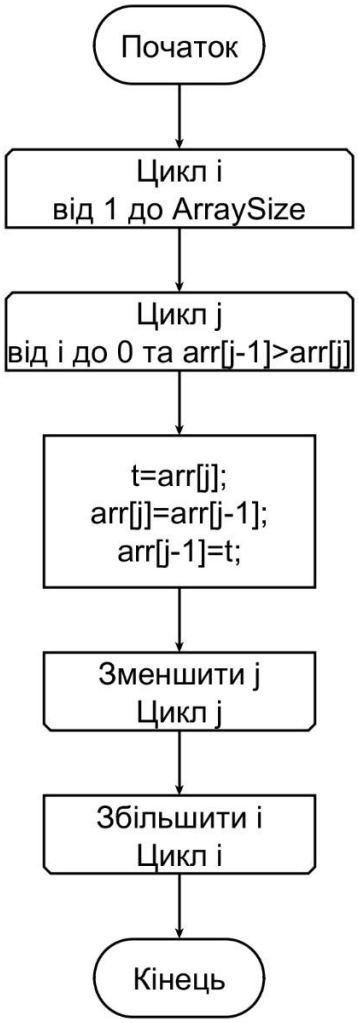
Схема алгоритму сортування бульбашкою



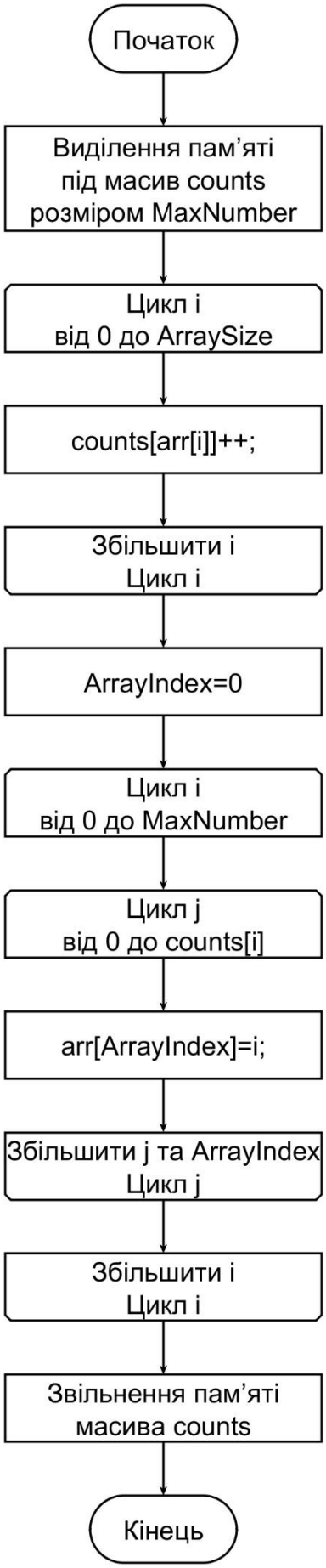
Сортування вибором



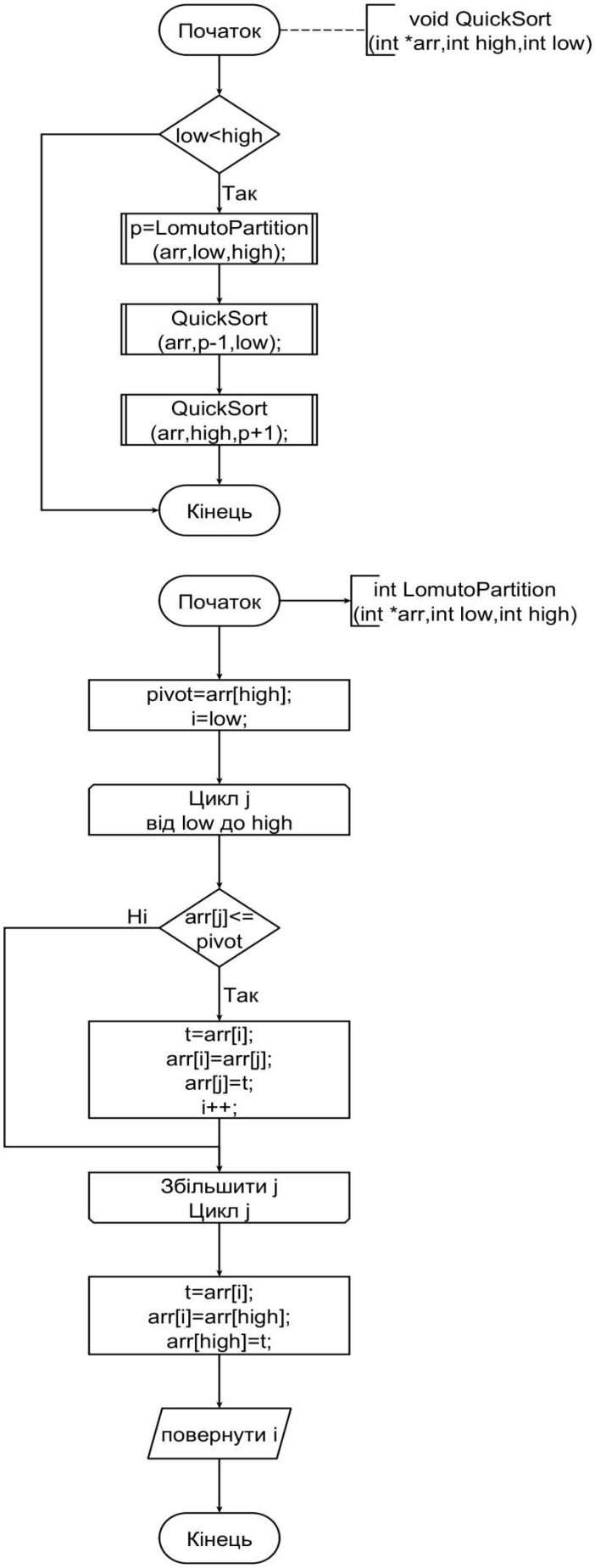
Сортування вставками



Сортування підрахунками



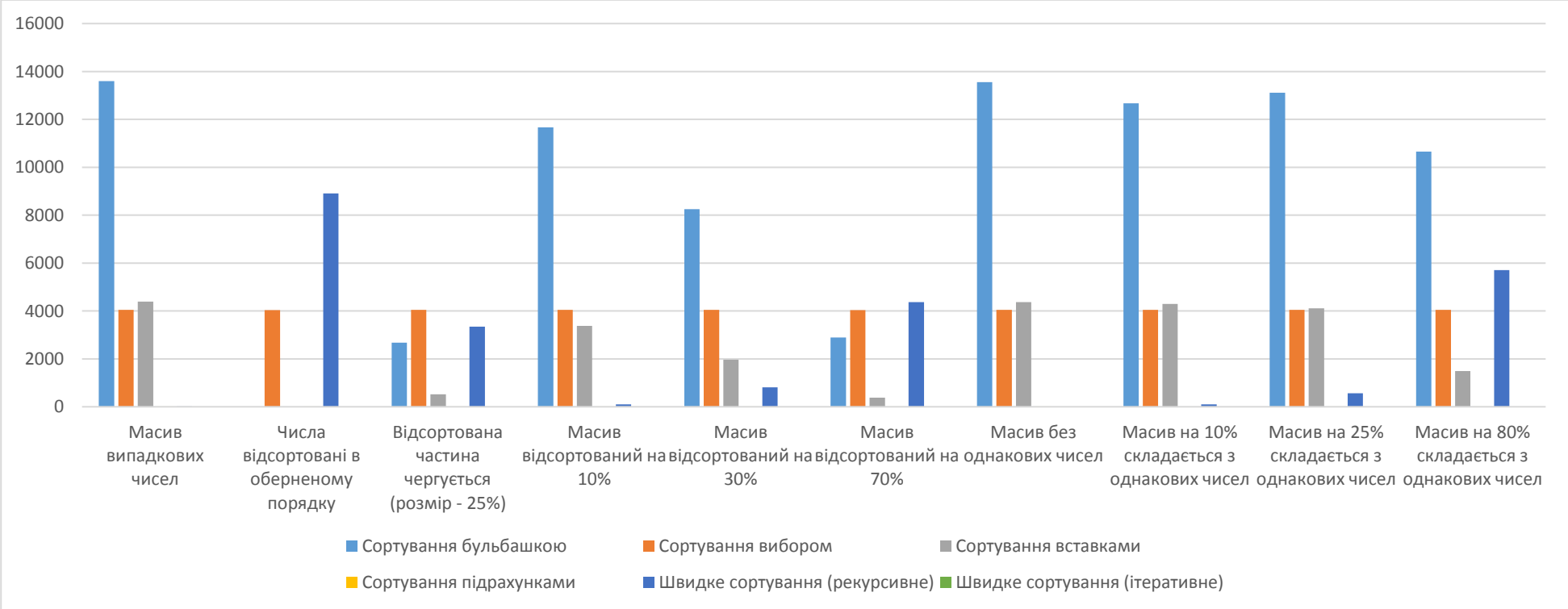
Швидке сортування (рекурсія)



Порівняння складності реалізованих алгоритмів

Назва алгоритму	Гірший випадок	Найкращий випадок
Сортування бульбашкою	$O(n^2)$	O(n)
Сортування вибором	$O(n^2)$	$O(n^2)$
Сортування вставками	$O(n^2)$	O(n)
Сортування підрахунками	O(n+max-min)	O(n+max-min)
Швидке сортування	$O(n^2)$	O(n log n)

Порівняння швидкодії (кількість елементів – 40000, час вказаний у мілісекундах)



Висновок. У процесі виконання даної роботи нами були розроблені процедури, які генерують дані різного характеру: випадкові, в різній мірі відсортовані та з різним процентом вмісту однакових значень. Для тестування реалізованих рішень була розроблена процедура, яка перевіряє вихідний масив на предмет наявності всіх чисел у початковому джерелі та коректності порядку розміщення даних.

Нами було реалізовано рекурсивний та ітеративний алгоритми швидкого сортування. Рекурсія має серйозний недолік, пов'язаний з переповненням стеку викликів операційної системи, що може спричинити некоректне завершення програми при спробі відсортувати більше ніж приблизно 40000 елементів, особливо якщо всі вони первісно розміщені в оберненому порядку. Ітеративна оптимізована авторська версія значно краща та найбільш швидкодіюча з усіх досліджених алгоритмів. Найгіршим алгоритмом в усіх випадках є метод бульбашки, далі з перемінним успіхом слідують сортування вибором і вставками. Згідно з наданим графіком можна зробити висновок, що «зручний» характер даних не завжди позитивно впливає на швидкість роботи алгоритму. Можна стверджувати, що сортування вибором, відноситься до таких алгоритмів, яке при будь-яких числах показує майже однакову продуктивність. Більш кращим в цьому плані є алгоритм сортування вставками.

ПРАКТИЧНА РОБОТА №5. Алгоритми сортування. Методи сортування великих обсягів даних.

Постановка задачі: дослідження алгоритмів плавного сортування та сортування злиттям.

Схема рекурсивного алгоритму сортування злиттям

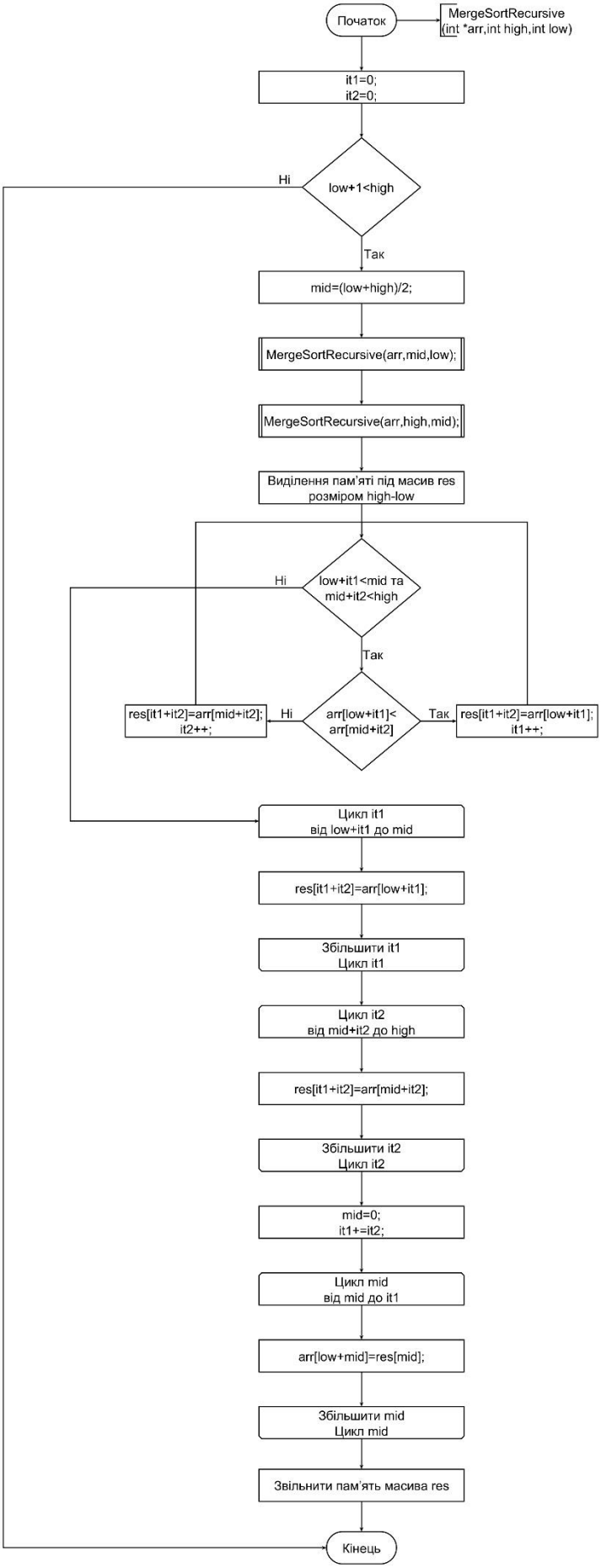
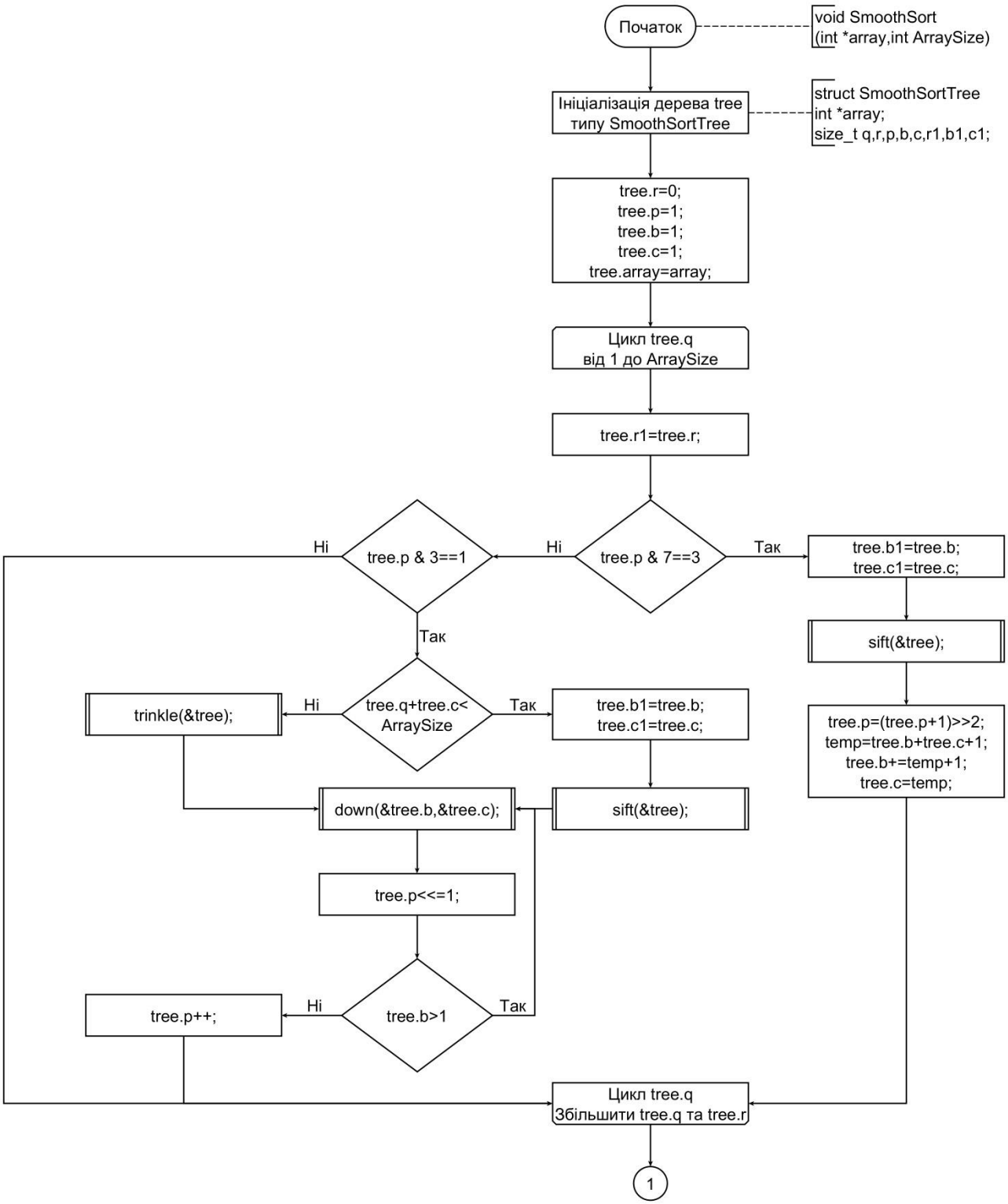
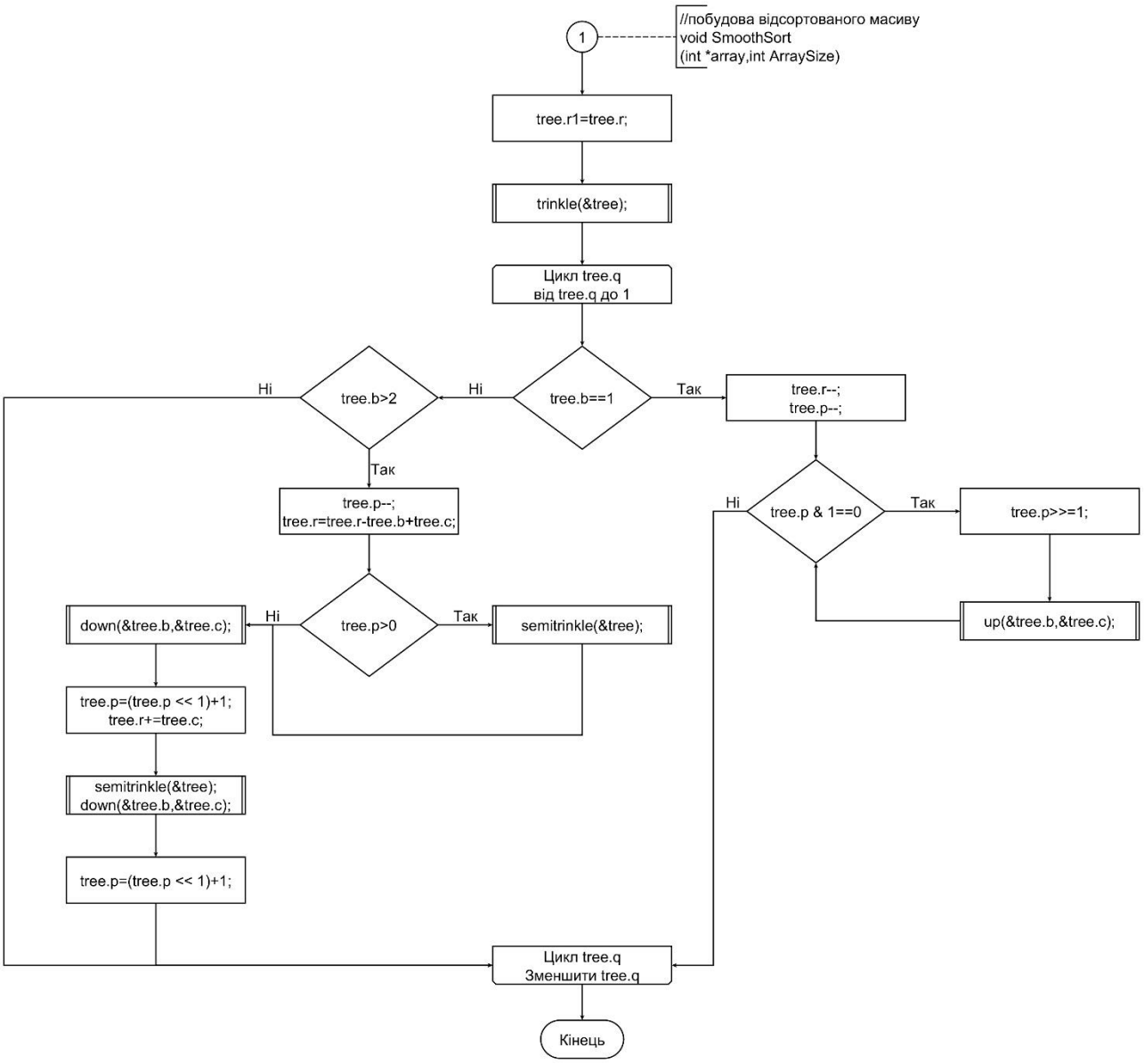
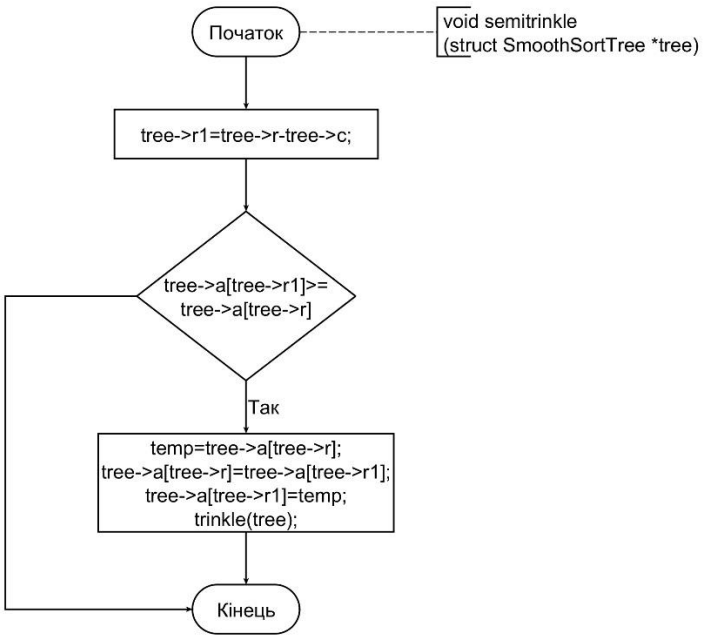


Схема алгоритму плавного сортування

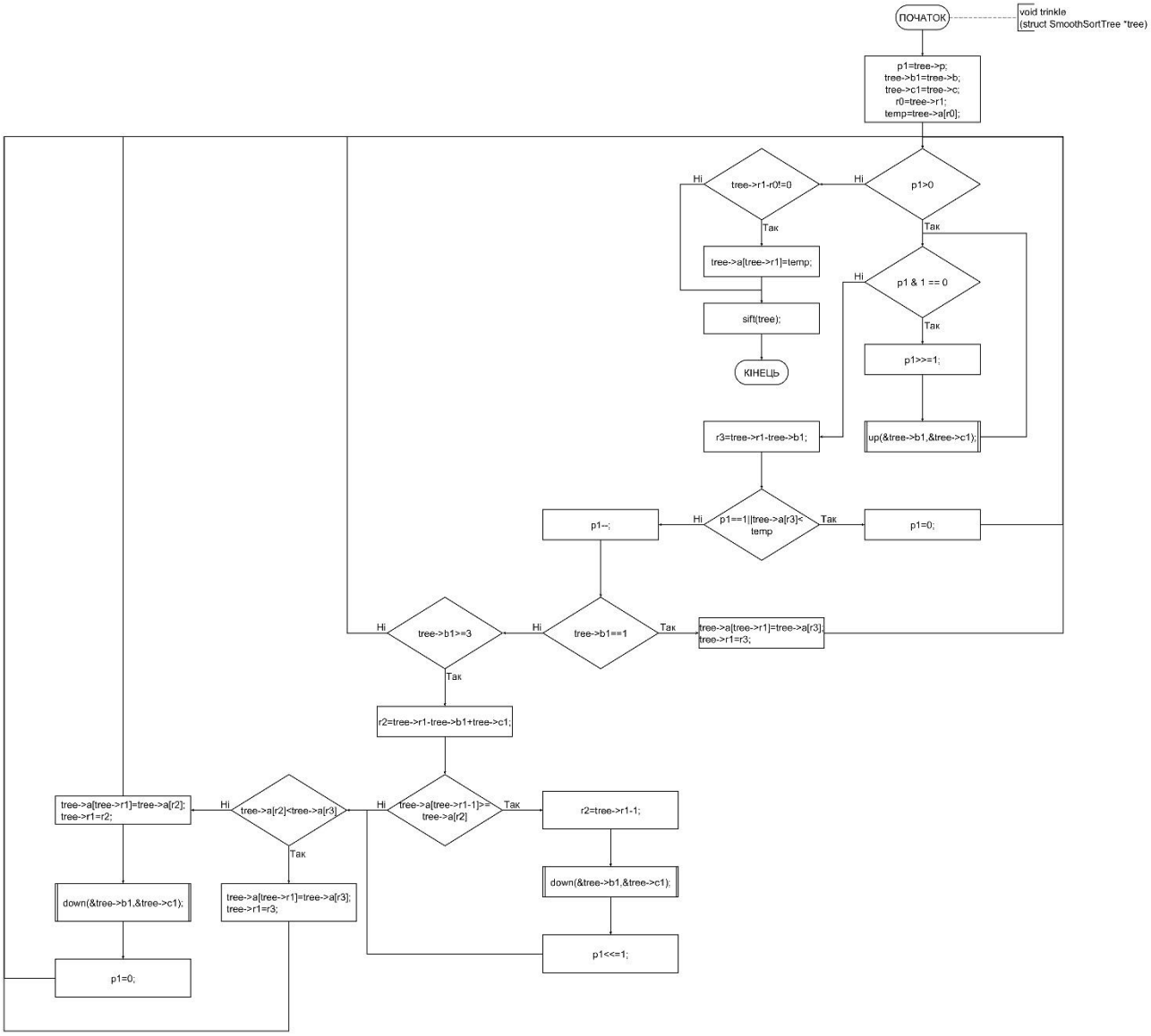




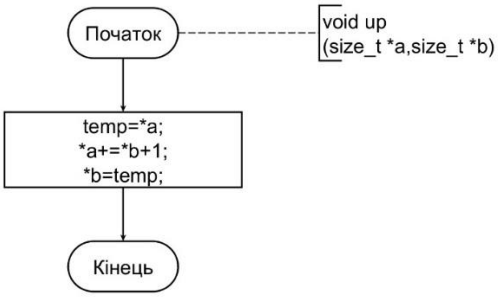
Процедура semitrinkle

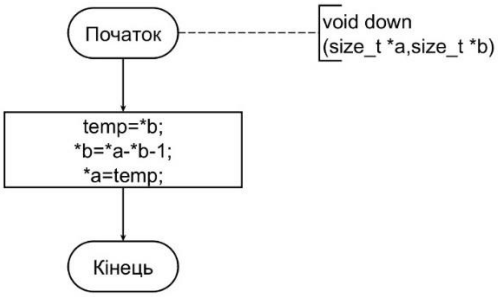


Процедура trinkle

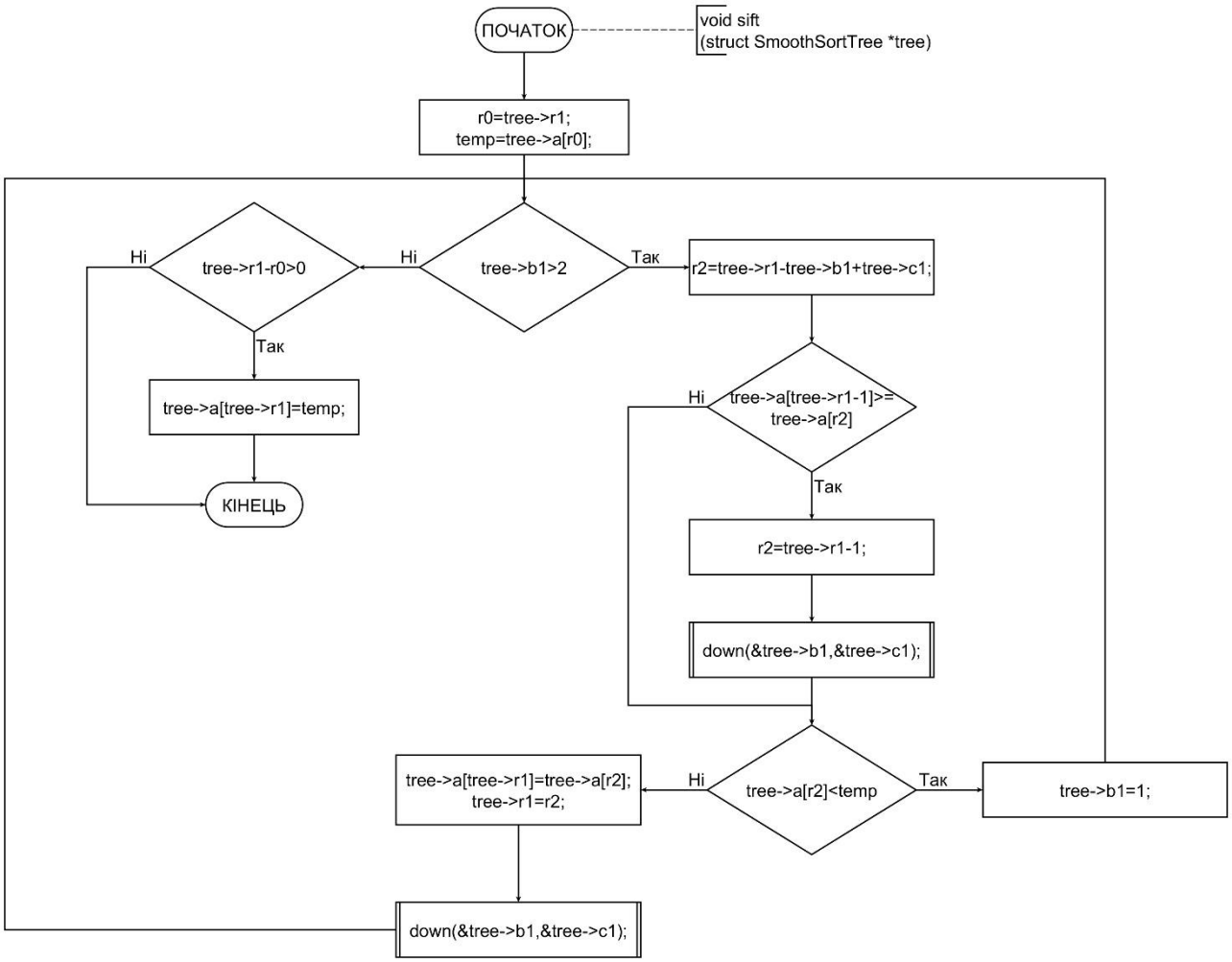


Процедури Up та Down





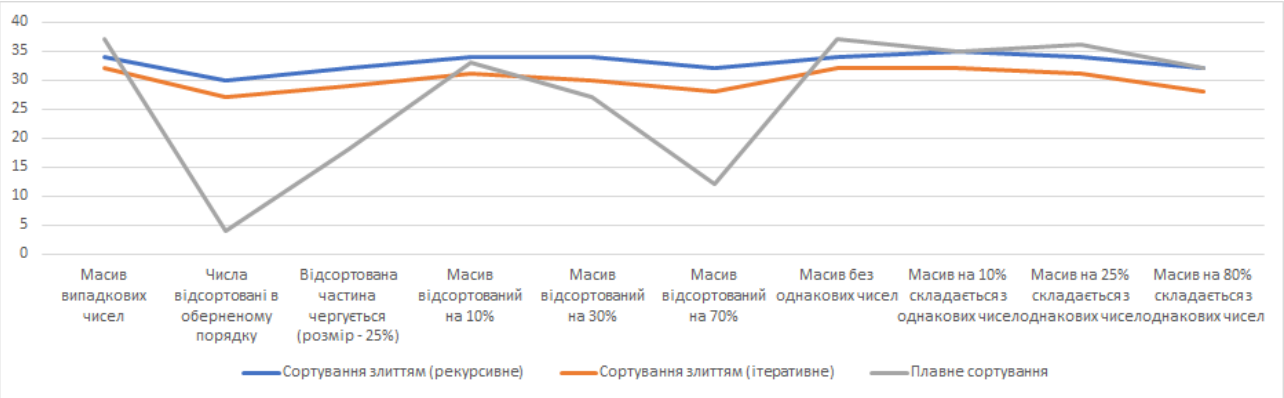
Процедура sift



Складність реалізованих алгоритмів

Назва алгоритму	Найгірший випадок	Середній випадок	Найкращий випадок
Сортування злиттям	n*log (n)	n*log (n)	n*log (n)
Плавне сортування	n*log (n)	n*log (n)	n

Порівняння швидкодії реалізованих алгоритмів (час – у мілісекундах)



Висновок. У процесі дослідження було встановлено, що швидкодія ітеративного та рекурсивного алгоритмів сортування злиттям знаходяться майже на одному рівні та не є суттєво залежними від характеру даних. Ефективність роботи плавного сортування сильно залежить від вхідних чисел: масив, відсортований в оберненому порядку оброблюється найбільш ефективно, а випадковий – найменш ефективно.

6. Відповіді на контрольні питання.

1. Перерахуйте та прокоментуйте переваги та недоліки швидкого сортування. Переваги:

- Один з самих швидкодіючих (на практиці) з алгоритмів внутрішнього сортування загального призначення.
- Простий в реалізації.
- Вимагає лише O(lg n) додаткової пам'яті для своєї роботи. (Не покращений рекурсивний алгоритм в гіршому випадку O(n) пам'яті)
- Добре поєднується з механізмами кешування і віртуальної пам'яті.
- Існує ефективна модифікація (алгоритм Седжвіка) для сортування рядків - спочатку порівняння з опорним елементом тільки по нульовому символу рядка, далі застосування аналогічної сортування для "більшого" і "меншого" масивів теж по нульовому символу, і для "рівного" масиву по вже першого символу.
- Працює на пов'язаних списках та інших структурах з послідовним доступом.

Недоліки:

- Сильно деградує по швидкості (до $\Theta(n^2)$). При невдалих виборах опорних елементів, що може трапитися при невдалих вхідних даних. Цього можна уникнути, використовуючи такі модифікації алгоритму,

як Introsort, або ймовірносно, вибираючи опорний елемент випадково, а не фіксованим чином.

- Наївна реалізація алгоритму може привести до помилки переповнення стека, так як їй може знадобитися зробити O(n) вкладених рекурсивних викликів. У покращених реалізаціях, в яких рекурсивний виклик відбувається тільки для сортування меншою з двох частин масиву, глибина рекурсії гарантовано не перевищить O(lg n).
- Нестійкий - якщо потрібна стійкість, доводиться розширювати ключ.

2. Перерахуйте та прокоментуйте переваги та недоліки сортування злиттям. Які найпоширеніші модифікації цього методу сортування?

Переваги:

- працює навіть на структурах даних послідовного доступу;
- добре поєднується з підкачкою та кешуванням пам'яті;
- непогано працює в паралельному варіанті: легко розбити завдання між процесорами порівну, але важко зробити так, щоб інші процесори взяли на себе роботу, в разі якщо один процесор затримається;
- не має «важких» вхідних даних;
- стійкий — зберігає порядок рівних елементів (що належать одному класу еквівалентності в порівнянні).

Недоліки:

- на «майже відсортованих» масивах працює так само довго, як на хаотичних. Існує варіант сортування злиттям, який працює швидше на частково впорядкованих даних, але він вимагає додаткової пам'яті, в доповненні до тимчасового буферу, який використовується безпосередньо для сортування.
- вимагає додаткової пам'яті за розміром вихідного масиву.

3. Перерахуйте та прокоментуйте переваги та недоліки пірамідального сортування.

Переваги:

- час роботи в найгіршому випадку – O(n log n);
- вимагає O(1) додаткової пам'яті.

Недоліки:

- нестійкий — для забезпечення стійкості потрібно розширювати ключ;
- на майже відсортованих даних працює так само довго, як і на хаотичних даних;
- складний в реалізації;
- на одному кроці вибірку доводиться робити хаотично по всій довжині масиву — тому алгоритм погано поєднується з кешуванням і (рос. "файлом підкачки", укр. "файлом довантаження") — віртуальної пам'яті;
- методу потрібно «миттєвий» прямий доступ; не працює на зв'язаних списках та інших структурах пам'яті послідовного доступу.

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«Київський Політехнічний Інститут імені Ігоря Сікорського»
Кафедра Технічної кібернетики

ЗВІТНІСТЬ ПО ІНДИВІДУАЛЬНІЙ РОБОТІ З ДИСЦИПЛІНИ «ПРОЕКТУВАННЯ АЛГОРИТМІВ»

Виконав студент
групи ЗПІ-зп-02
Кононов М. А.

Мова програмування: C

Київ-2021

1. **Постановка задачі:**

На основі карти України обрати частину території, яка включає в себе населений пункт проживання студента, населений пункт його рідних (не співпадає з попереднім) та ще декілька населених пунктів навколо них. На основі виділеної території створити базовий граф, вершини якого – населені пункти, ребра – дороги між ними. Кількість вершин повинна бути від 15 до 20, кількість ребер від 30 . Вагами ребер необхідно взяти:

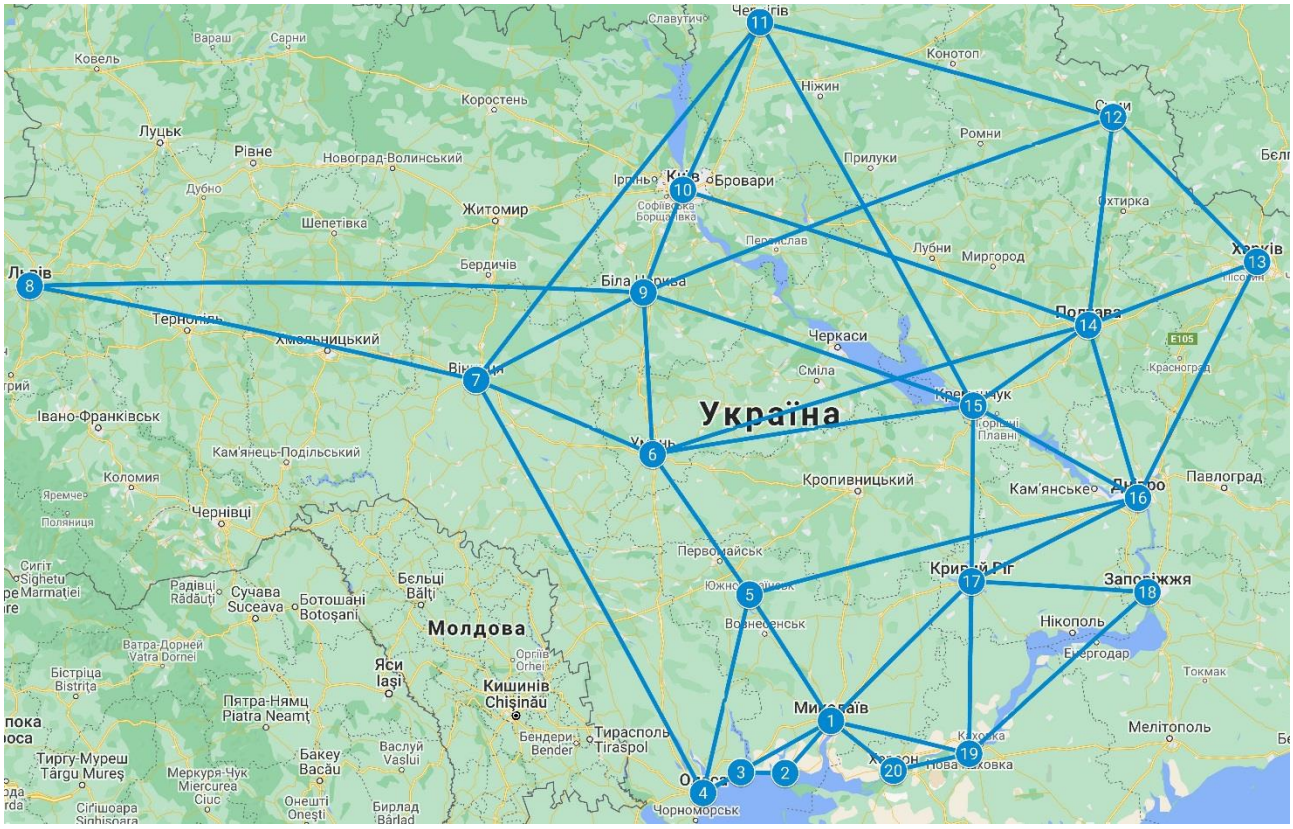
a) довжину шляху між населеними пунктами в км,

b) тип дороги, ставлячи йому відповідний пріоритет.

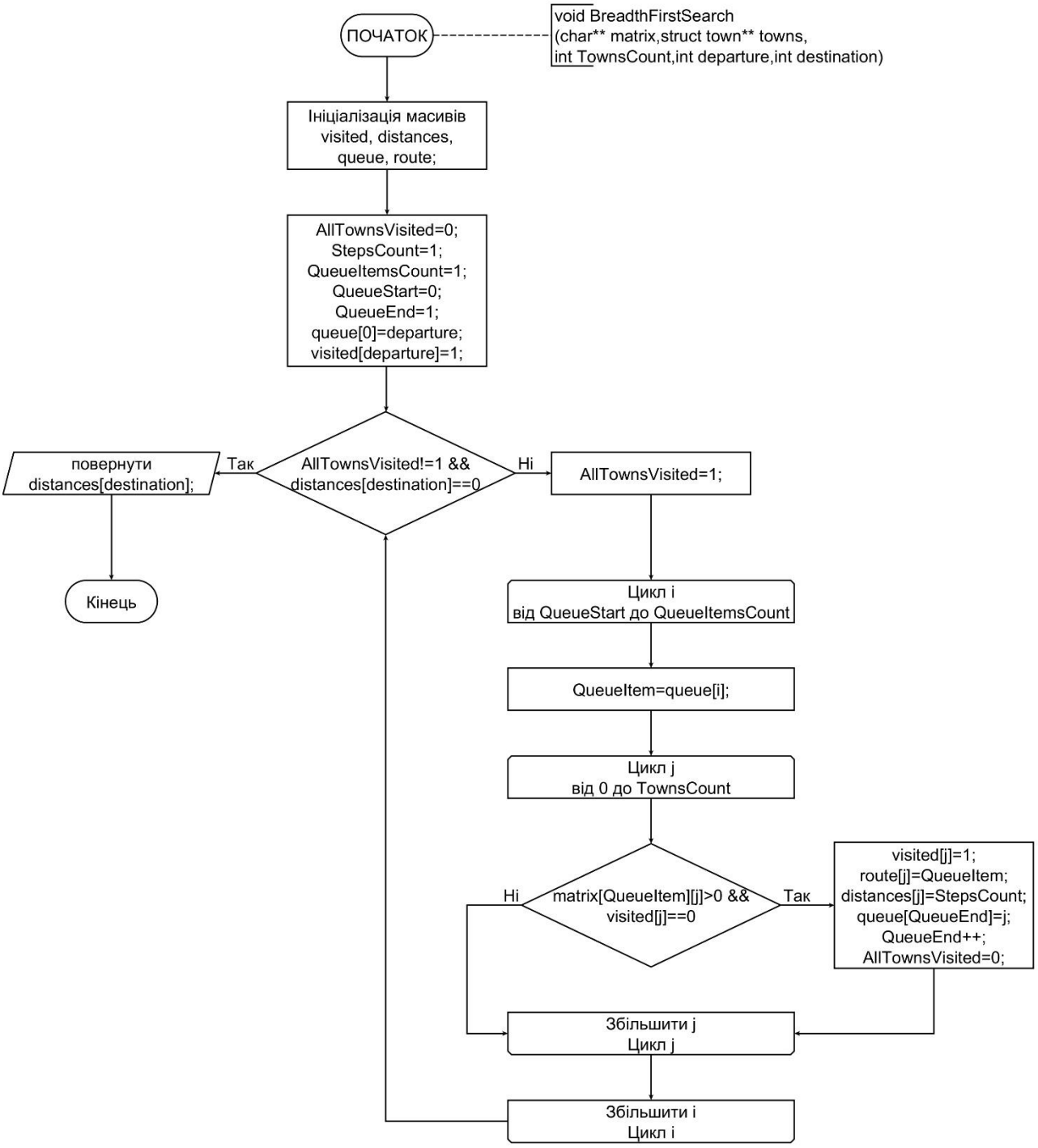
Для кожного населеного пункту визначити його координати та кількість населення.

2. **Скріншот частини території України, що застосовується в роботі.**

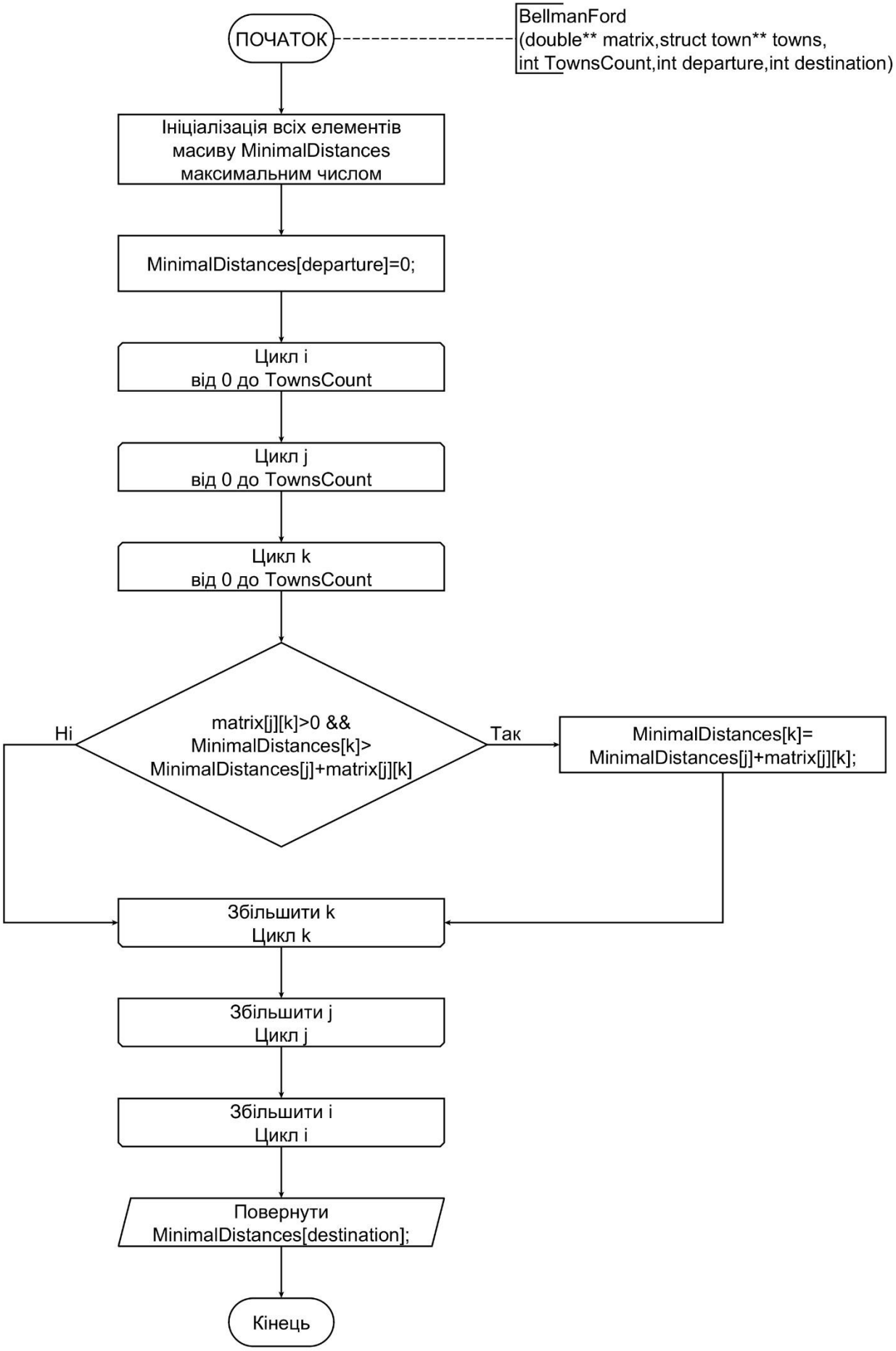
3. Базовий граф:


Україна
Молдова
Київ
Львів
Одеса
Харків
Дніпро
Миколаїв
Кривий Ріг
Запоріжжя
Полтава
Вінниця
Житомир
Луцьк
Рівне
Чернівці
Івано-Франківськ
Тернопіль
Хмельницький
Черкаси
Кропивницький
Кишинів
Iași
Cernăuți
Suceava
Botoșani
Bălți
Tiraspol
Bender
Cahul
Ismail
Мелітополь
Нікополь
Енергодар
Токмак
Чорноморськ
Ніжин
Конотоп
Суми
Ромни
Прилуки
Бровари
Миргород
Лубни
Кременчук
Сміла
Кам'янське
Павлоград
Первомайськ
Южноукраїнськ
Вознесенськ
Новоукраїнка
Кіровоград
Житомир
Бердичів
Біла Церква
Коростень
Новоград-Волинський
Шепетівка
Дубно
Ковель
Вараш
Сарни
Славутич
Чернігів
Полтава
Харків
Сєвєрод
Лубни
Миргород
Черкаси
Сміла
Кам'янське
Кропивницький
Первомайськ
Южноукраїнськ
Вознесенськ
Кривий Ріг
Запоріжжя
Нікополь
Енергодар
Токмак
Мелітополь
Каховка
Новокаховка
Чорноморськ
Одеса
Миколаїв
Кіровоград
Житомир
Бердичів
Біла Церква
Хмельницький
Вінниця
Тернопіль
Івано-Франківськ
Коломия
Чернівці
Луцьк
Рівне
Новоград-Волинський
Дубно
Шепетівка
Коростень
Ковель
Вараш
Сарни
Славутич
Чернігів
Конотоп
Ніжин
Ромни
Прилуки
Бровари
Миргород
Лубни

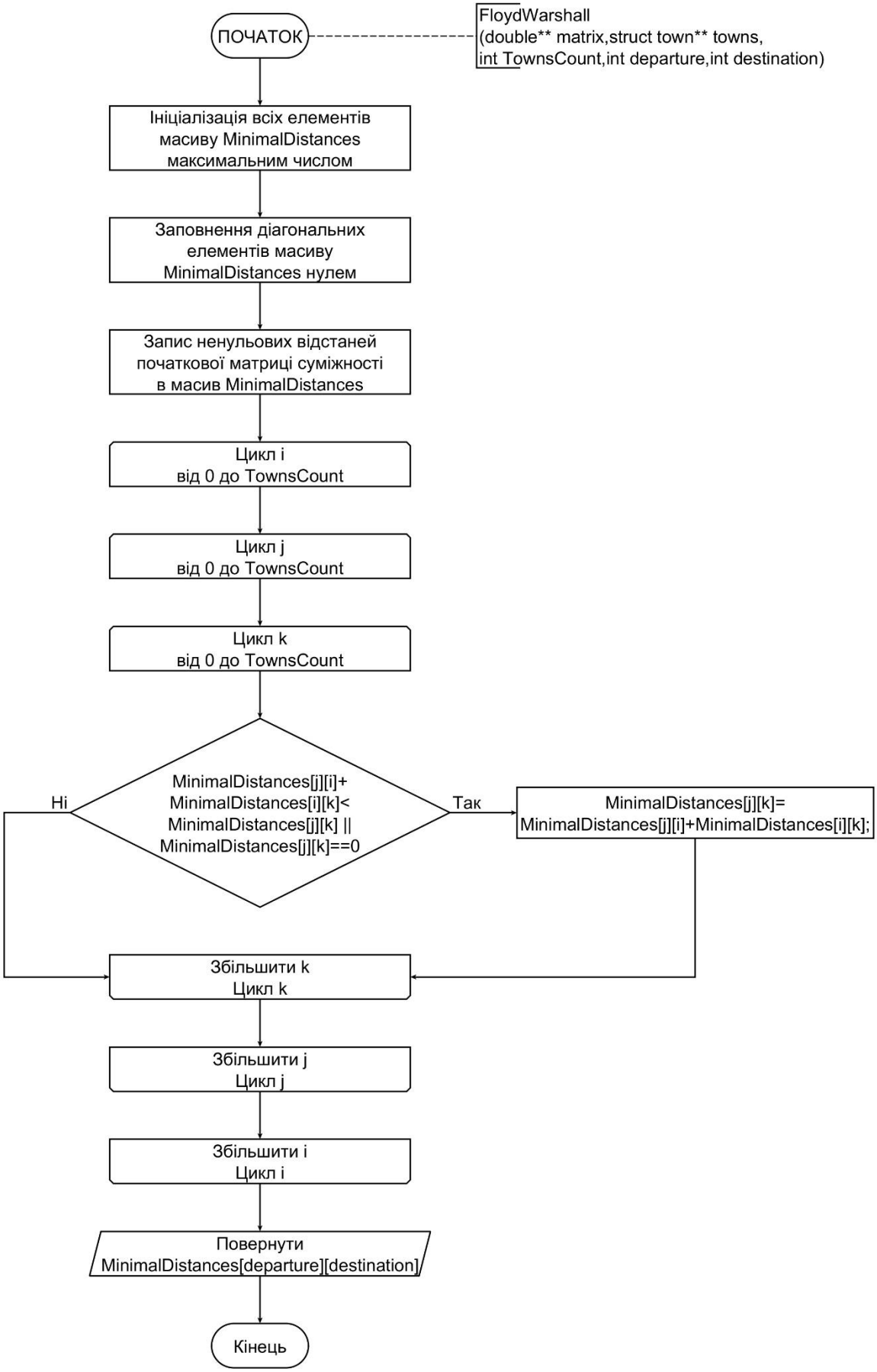
Пошук в ширину



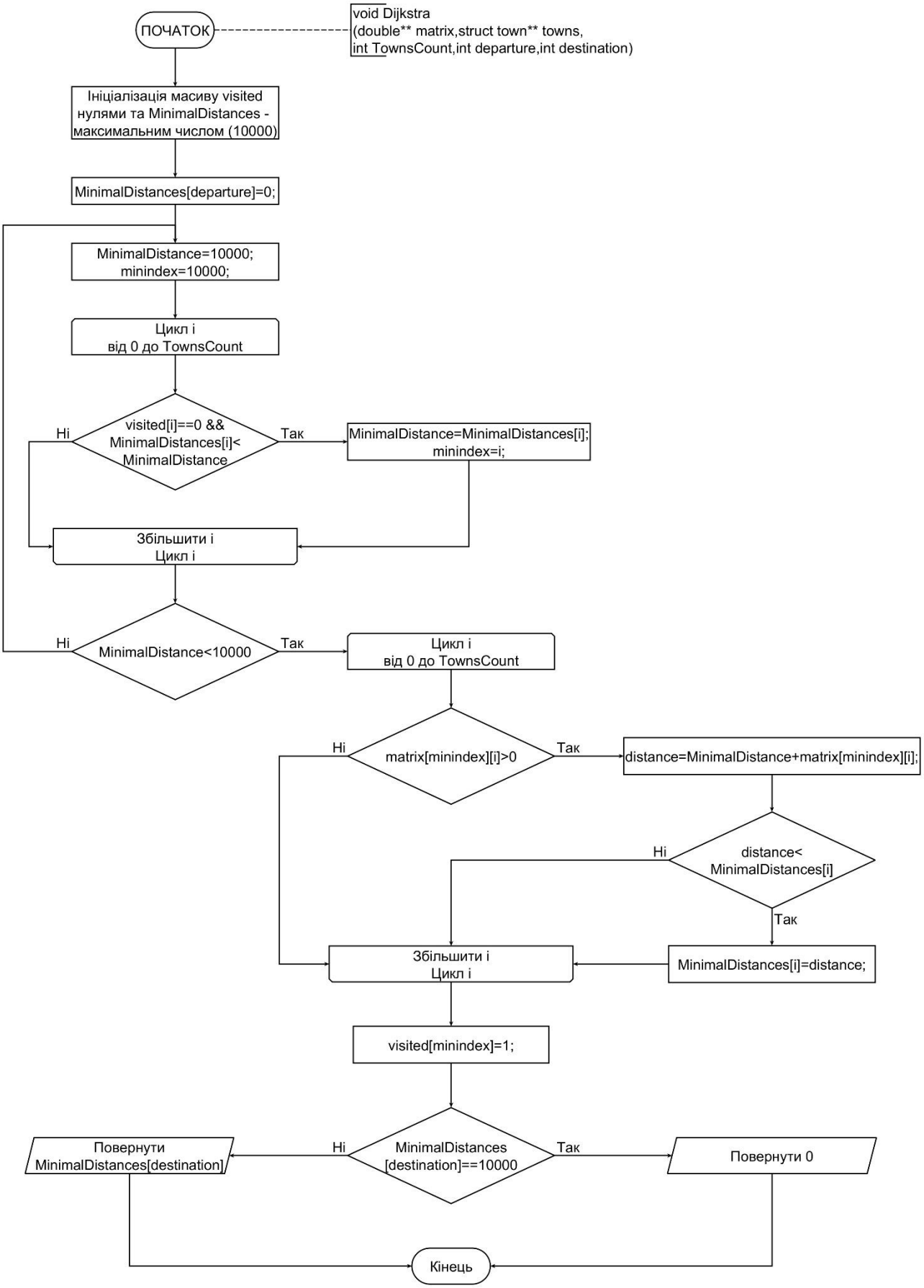
Алгоритм Беллмана-Форда



Алгоритм Флойда-Уоршелла



Алгоритм Дейкстри



5. Розв’язок наступних задач:

А) За допомогою одного з алгоритмів обходу графів обійти всі населені пункти на виділеній територій. Початком вважати населений пункт проживання студента, кінцевою точкою – пункт проживання його рідних. На довжину шляху та тип дороги не зважати.

Для обходу графу використовується рекурсивний алгоритм в глибину.

```
C:\Windows\system32\cmd.exe
1. Visit all towns starting the route from Mykolaiv;
2. Find a route without taking a distance into account;
3. Find the shortest route;
4. Show towns from the west to the east;
5. Show towns from the north to the south;
6. Show towns according to the population;
7. Test random graphs;
8. Exit.
Your choice: 1
Mykolaiv->Ochakiv->Yuzhne->Odessa->Yuzhnoukrainsk->Uman->Vinnytsia->Lviv->Bila T
serkva->Kyiv->Chernihiv->Sumy->Kharkiv->Poltava->Kremenchuk->Dnipro->Kryvyi Rih-
>Zaporizhzhia->Nova Kakhovka->Kherson
```

Б) За допомогою одного з алгоритмів знайти шлях від населеного пункту проживання студента до пункту проживання його рідних. На довжину шляху та тип дороги не зважати.

Для пошуку найменшої кількості кроків використовуються алгоритми пошуку в ширину та глибину:

```
Number of a departure town:
 0.       Mykolaiv;  1.        Ochakiv;  2.         Yuzhne;  3.         Odessa;
 4. Yuzhnoukrainsk;  5.           Uman;  6.      Vinnytsia;  7.           Lviv;
 8.   Bila Tserkva;  9.           Kyiv; 10.      Chernihiv; 11.           Sumy;
12.        Kharkiv; 13.        Poltava; 14.     Kremenchuk; 15.         Dnipro;
16.     Kryvyi Rih; 17.   Zaporizhzhia; 18.  Nova Kakhovka; 19.        Kherson;
20.           Exit. Your choice: 10
Number of a destination town:
 0.       Mykolaiv;  1.        Ochakiv;  2.         Yuzhne;  3.         Odessa;
 4. Yuzhnoukrainsk;  5.           Uman;  6.      Vinnytsia;  7.           Lviv;
 8.   Bila Tserkva;  9.           Kyiv; 10.      Chernihiv; 11.           Sumy;
12.        Kharkiv; 13.        Poltava; 14.     Kremenchuk; 15.         Dnipro;
16.     Kryvyi Rih; 17.   Zaporizhzhia; 18.  Nova Kakhovka; 19.        Kherson;
20.           Exit. Your choice: 0
Chernihiv - Mykolaiv;
DEPTH-FIRST SEARCH: minimal count of steps is 3;
route: Chernihiv->Vinnytsia->Odessa->Mykolaiv
--------------------------------------------------------------------------------
BREADTH FIRST SEARCH: minimal count of steps is 3
route: Chernihiv->Vinnytsia->Odessa->Mykolaiv
--------------------------------------------------------------------------------
```

В) За допомогою одного з алгоритмів знайти найкоротший шлях для двох випадків обирання критерію проходження (вага a) та b)) від населеного пункту проживання студента до пункту проживання його рідних. Порівняти результати.

Для вирішення цього завдання нами використовуються три алгоритми: Дейкстри, Форда-Беллмана та Флойда-Уоршелла:

```
C:\Windows\system32\cmd.exe
Type of road:
0. Direct route;
1. The shortest highway;
2. The longest highway;
3. Average length of all highways;
4. The shortest foot path;
5. The longest foot path;
6. Average length of all foot paths;
7. The shortest railway;
8. The longest railway;
9. Average length of all railways;
Your choice: 7
BELLMAN-FORD ALGORITHM:
minimal distance from Mykolaiv to Kharkiv is 591.00 km
route: Mykolaiv->Kryvyi Rih->Dnipro->Kharkiv
--------------------------------------------------------------------------------
FLOYD-WARSHALL ALGORITHM:
minimal distance from Mykolaiv to Kharkiv is 591.00 km
route: Mykolaiv->Kryvyi Rih->Dnipro->Kharkiv
--------------------------------------------------------------------------------
DIJKSTRA'S ALGORITHM:
minimal distance from Mykolaiv to Kharkiv is 591.00 km
route: Mykolaiv->Kryvyi Rih->Dnipro->Kharkiv
--------------------------------------------------------------------------------
```

З Одеси немає залізничного шляху до Очакову:

```
Type of road:
0. Direct route;
1. The shortest highway;
2. The longest highway;
3. Average length of all highways;
4. The shortest foot path;
5. The longest foot path;
6. Average length of all foot paths;
7. The shortest railway;
8. The longest railway;
9. Average length of all railways;
Your choice: 9
BELLMAN-FORD ALGORITHM:
there is no route from Odessa to Ochakiv.
FLOYD-WARSHALL ALGORITHM:
there is no route from Odessa to Ochakiv.
DIJKSTRA'S ALGORITHM:
there is no route from Odessa to Ochakiv.
```

Г) Зробити сортування населених пунктів на основі відомих алгоритмів за їх координатами із заходу на схід, та кількістю населення з більшого к меншому.

Задача вирішувалась за допомогою сортування вставками. Населені пункти із заходу на схід:

```
Your choice: 4
 0.          Lviv;  1.      Vinnytsia;  2.   Bila Tserkva;  3.          Uman;
 4.          Kyiv;  5.         Odessa;  6.         Yuzhne;  7. Yuzhnoukrainsk;
 8.     Chernihiv;  9.        Ochakiv; 10.       Mykolaiv; 11.        Kherson;
12.  Nova Kakhovka; 13.     Kryvyi Rih; 14.     Kremenchuk; 15.        Poltava;
16.          Sumy; 17.         Dnipro; 18.   Zaporizhzhia; 19.        Kharkiv;
```

Населені пункти з півдня на північ:

```
Your choice: 5
 0.          Odessa;  1.         Ochakiv;  2.          Yuzhne;  3.         Kherson;
 4.   Nova Kakhovka;  5.        Mykolaiv;  6. Yuzhnoukrainsk;  7.    Zaporizhzhia;
 8.      Kryvyi Rih;  9.         Dnipro; 10.           Uman; 11.      Kremenchuk;
12.       Vinnytsia; 13.         Poltava; 14.   Bila Tserkva; 15.            Lviv;
16.         Kharkiv; 17.            Kyiv; 18.           Sumy; 19.       Chernihiv;
```

Населені пункти розташовані за кількістю населення:

```
Your choice: 6
 0.            Kyiv;  1.        Kharkiv;  2.         Odessa;  3.         Dnipro;
 4.    Zaporizhzhia;  5.           Lviv;  6.      Kryvyi Rih;  7.        Mykolaiv;
 8.       Vinnytsia;  9.        Kherson; 10.        Poltava; 11.       Chernihiv;
12.            Sumy; 13.     Kremenchuk; 14.   Bila Tserkva; 15.            Uman;
16.   Nova Kakhovka; 17. Yuzhnoukrainsk; 18.         Yuzhne; 19.         Ochakiv;
```

Д) Для кожного попереднього пункту описати обраний алгоритм, його особливості, обґрунтувати вибір цього алгоритму та визначити його складність.

Алгоритм	**Складність**
Пошук в глибину	O(m)
Пошук в ширину	O(V+E)
Алгоритм Дейкстри	O(n log n + m log n)=O(m log n)
Алгоритм Форда-Беллмана	Найгірший випадок: O(VE); найкращий: O(E)
Алгоритм Флойда-Уоршелла	$O(V^3)$

Для пошуку найменшої кількості кроків з однієї вершини в іншу без урахування відстані нами було реалізовано алгоритми в глибину та ширину. Пошук в глибину є рекурсивним і основна його суть полягає в тому, що ми рухаємось в глиб графу доки це можливо і фактично здійснюємо вибір серед всіх варіантів шляхів, вибираючи найкоротший. Він є найменш ефективним і дуже повільно працює на сильно зв’язних графах, але добре підходить для дерев (незв’язних графів). Пошук в ширину є більш швидким в усіх випадках, оскільки заснований на ідеї динамічного програмування. Його суть полягає в тому, що ми дивимось навколо, запам’ятовуючи найменшу кількість кроків з кожної вершини до сусідніх.

Для пошуку найкоротших шляхів з урахуванням відстані ми використали три алгоритми: Дейкстри, Форда-Беллмана та Флойда-Воршелла.

Алгоритм Флойда-Воршелла порівнює всі можливі шляхи в графі між кожною парою вершин. Він виконується за $\Theta(|V|^3)$ порівнянь. Це доволі

примітивно, враховуючи, що в графі може бути до $\Omega(|V|^2)$ ребер, і кожну комбінацію буде перевірено. Він виконує це шляхом поступового поліпшення оцінки по найкоротшому шляху між двома вершинами, поки оцінка не стає оптимальною.

Розгляньмо граф G з ребрами V, пронумерованими від 1 до N. Крім того розгляньмо функцію shortestPath(i, j, k), яка повертає найкоротший шлях від i до j, використовуючи вершини з множини {1,2,…,k} як внутрішні у шляху. Тепер, маючи таку функцію, нам потрібно знайти найкоротший шлях від кожного i до кожного j, використовуючи тільки вершини від 1 до k + 1.

Для кожної з цих пар вершин, найкоротший шлях може бути або (1)-шлях, у якому є тільки вершини з множини {1, …, k}, або (2)- шлях, який проходить від i до k + 1 а потім від k + 1 до j. Найкоротший шлях від i to j that only uses vertices 1 через k визначається функцією shortestPath(i, j, k), і якщо є коротший шлях віді до (k + 1 до j), то довжина цього шляху буде сумою(конкатенацією) найкоротшого шляху віді до k + 1 (використовуючи вершини{1, …, k}) і найкоротший шлях від k + 1 до j (також використовуючи вершини з {1, …, k}).

Алгоритм Форда-Беллмана представляє з себе кілька фаз. На кожній фазі проглядаються всі ребра графа, і алгоритм намагається справити релаксацію (relax, ослаблення) уздовж кожного ребра (u, v) ваги w(u, v). Релаксація вздовж ребра — це спроба поліпшити значення v.d значенням v.u + w(u, v). Фактично це означає, що ми намагаємося поліпшити значення для вершини v, користуючись ребром (u, v) і поточним значенням для вершини u. Стверджується, що достатньо G. V-1 фази алгоритму, щоб коректно порахувати довжини всіх найкоротших шляхів у графі (цикли негативної ваги відсутні). Для недосяжних вершин відстань v. d залишиться нескінченністю.

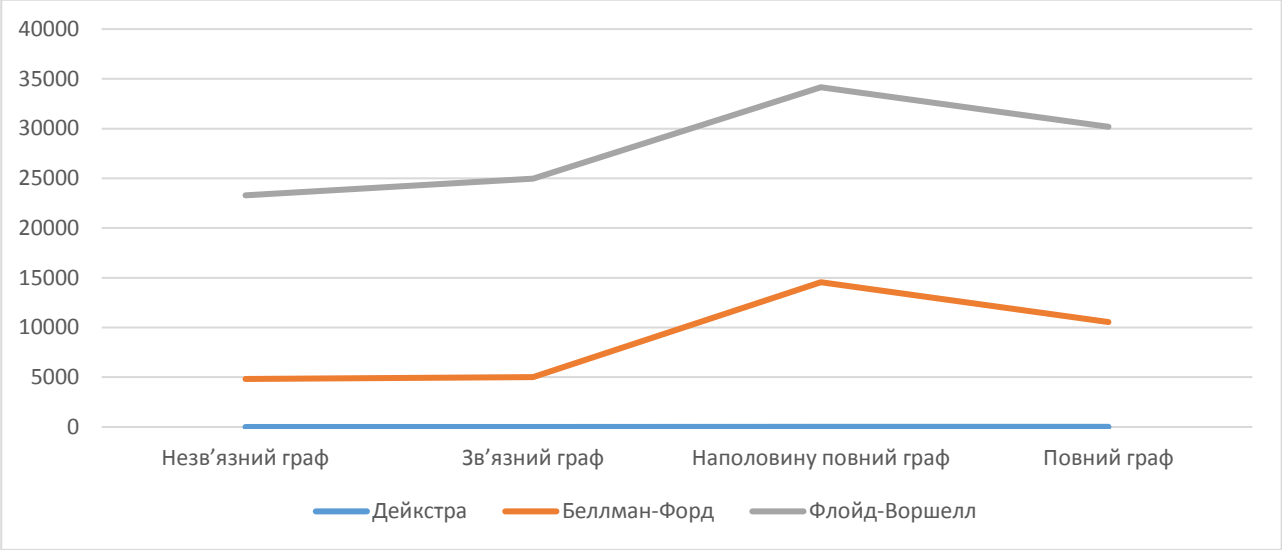
Найпростіша реалізація алгоритму Дейкстри потребує $O(V^2)$ дій. У ній використовується масив відстаней та масив позначок. На початку алгоритму відстані заповнюються великим додатнім числом (більшим максимального можливого шляху в графі), а масив позначок заповнюється нулями. Потім

відстань для початкової вершини вважається рівною нулю і запускається основний цикл.

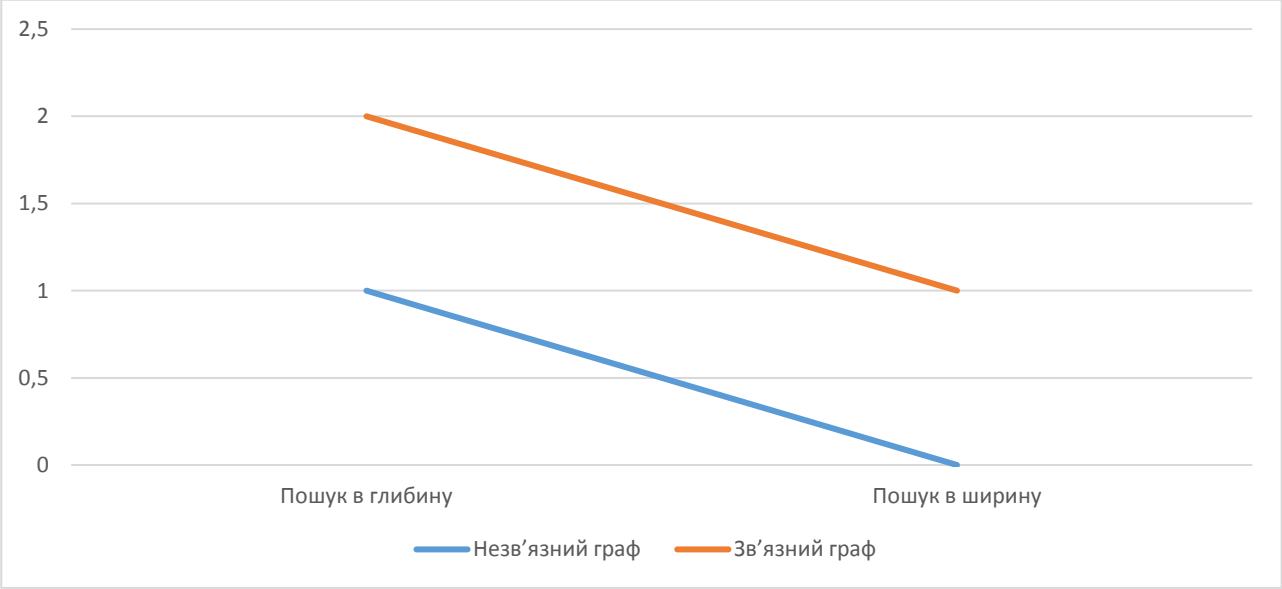
На кожному кроці циклу ми шукаємо вершину з мінімальною відстанню і прапором рівним нулю. Потім ми встановлюємо в ній позначку 1 і перевіряємо всі сусідні з нею вершини. Якщо в ній відстань більша, ніж сума відстані до поточної вершини і довжини ребра, то зменшуємо його. Цикл завершується коли позначки всіх вершин стають рівними 1.

Е) Тест швидкодії роботи всіх алгоритмів на кількох випадкових матрицях суміжності розміром 1000 на 1000 (час – у мілісекундах). Вказано середні результати серед 10 проведених експериментів пошуку відстані.

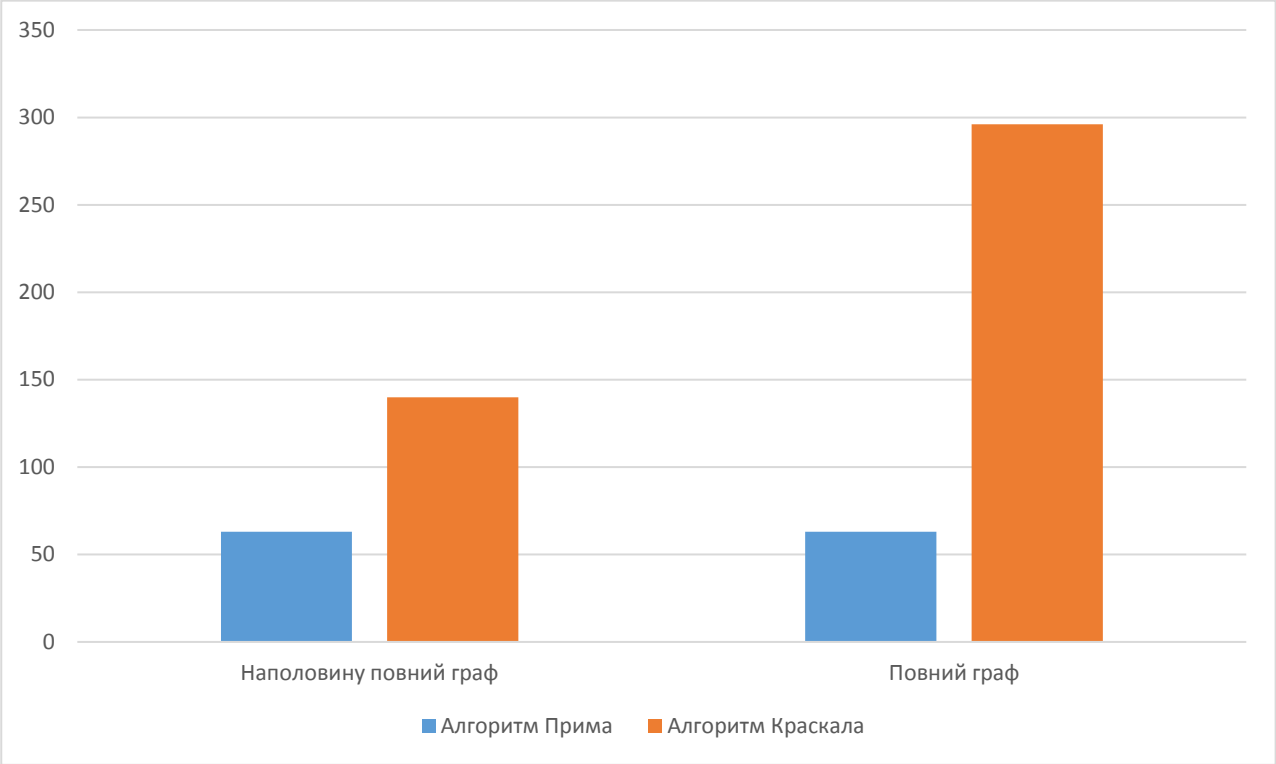
Пошук найкоротшої відстані



Пошук найменшої кількості кроків



Пошук остовного дерева (каркасу графа) мінімальної ваги



Загальний висновок. У процесі виконання роботи ми реалізували всі основні популярні алгоритми обходу та пошуку найкоротшої відстані і перевірили коректність їх роботи як на реальному невеликому графі (20 вершин та 40 ребер), який є зображенням частини території України, так і на випадкових графах різного ступеня зв'язності. Оскільки алгоритм пошуку в глибину дуже повільно працює на сильно зв'язних графах, експерименти з такими графами не проводились. Реалізовані нами алгоритми пошуку каркасу графа некоректно працюють з незв'язними графами, тому надано результати швидкодії роботи лише для наполовину повного та повного графів. У процесі дослідження було встановлено, що найбільш швидкодіючим алгоритмом пошуку найменшої кількості кроків з однієї вершини в іншу є пошук в ширину, а найбільш ефективним способом знаходження найменшої відстані є алгоритм Дейкстри. Найгіршим з досліджених алгоритмів є метод Флойда-Воршелла, який найкраще всього підходить для орієнтованих графів. Алгоритм Форда-Беллмана може працювати з від'ємними вагами, на відміну від найшвидшого алгоритму Дейкстри.